

Documentación Técnica — Sistema NAVES

Panel de Evaluación + Panel de Administración + Generador de Programación

INALDE Business School · Executive MBA

Alcance de este documento. Describe un **sistema genérico**, reutilizable para cualquier **cohorte** y cualquier **modalidad** del Executive MBA de INALDE. A lo largo del texto se usa una **cohorte de ejemplo** para ilustrar con datos concretos: **NAVES 2026 · Fin de Semana 24-26** (34 proyectos, 7 jornadas en 4 fechas de junio de 2026). Cuando aparezcan cifras, nombres o fechas específicas, son de esa cohorte de ejemplo — el sistema no está limitado a ellas. Ver §16 para el modelo multi-cohorte.

1. Contexto y propósito

NAVES es el trabajo de grado del **Executive MBA de INALDE Business School**. El MBA tiene **dos modalidades** según el ritmo de clases: **Intensivo** (clases aproximadamente una vez al mes, en tres días corridos de lunes a miércoles) y **Fin de Semana** (clases los viernes y sábados). En cada edición, los equipos de una cohorte presentan sus emprendimientos ante un panel de evaluadores externos (panelistas), en varias jornadas distribuidas en varias fechas.

Aclaración sobre fechas: lo que cambia entre modalidades es el ritmo de las clases, no un mes fijo. Las **presentaciones de NAVES** suelen caer en **noviembre** para el Intensivo y en **junio** para el Fin de Semana, pero las clases del programa se dictan a lo largo del año (mensuales en el Intensivo; cada fin de semana en el FS).

Códigos cortos: en la plataforma y en el código se usa **FS** = **Fin de Semana** e **INT** = **Intensivo** (ej. cohorte **fs-24-26**). Es solo un código; el nombre que se muestra al usuario es siempre "Intensivo" o "Fin de Semana".

Ejemplo (cohorte FS 24-26): 34 equipos presentan en **7 jornadas** distribuidas en **4 fechas** (12, 13, 19 y 20 de junio de 2026).

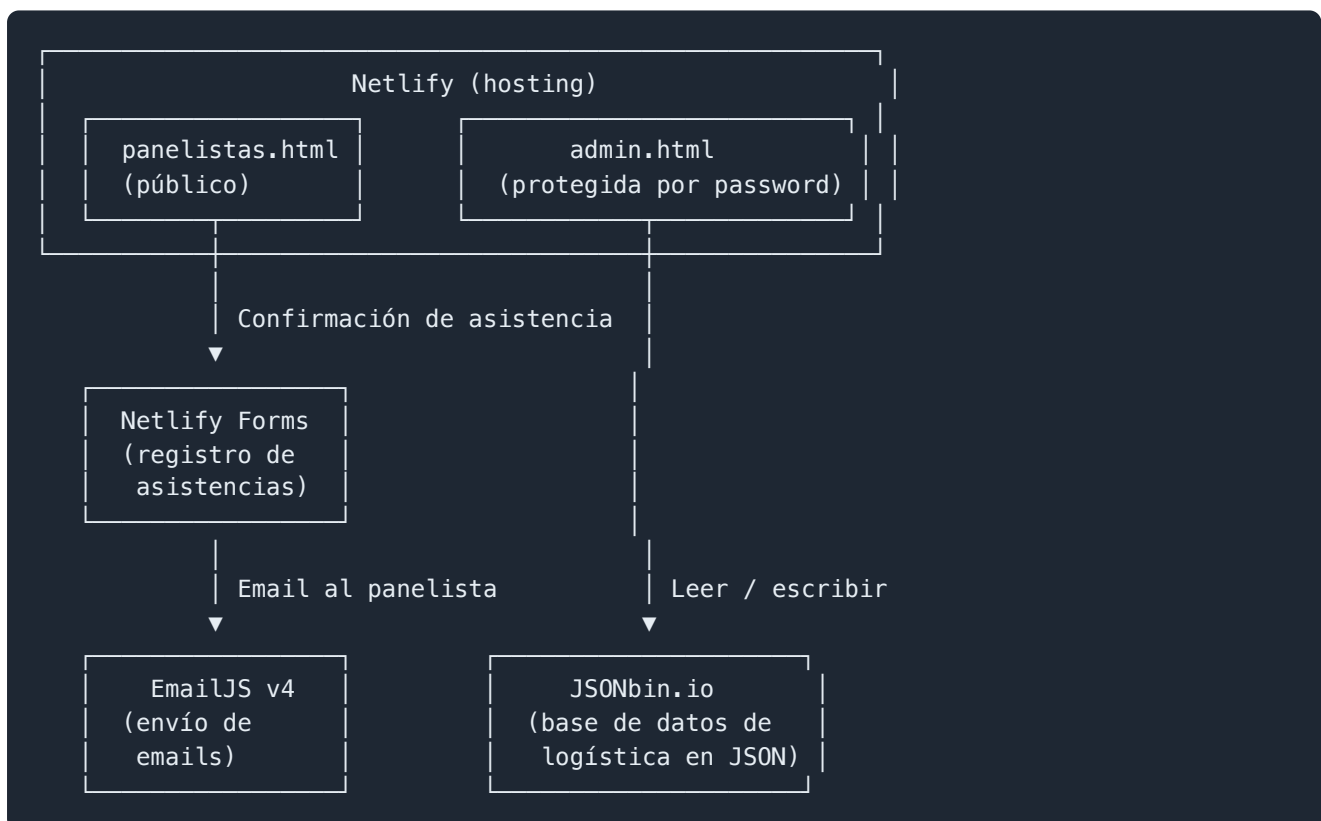
El sistema está compuesto por **cuatro páginas HTML independientes** que resuelven problemas concretos:

Archivo	Público	Función
<code>index.html</code>	Equipo interno INALDE	Base de datos de proyectos: posts LinkedIn, one pagers, programación (ver §6)
<code>panelistas.html</code>	Panelistas invitados	Ver proyectos, confirmar asistencia, declarar preferencias logísticas (ver §7)
<code>admin.html</code>	Coordinador INALDE	Gestionar logística, enviar emails de confirmación, exportar resumen (ver §8)
<code>programador.html</code>	Coordinador / profesor	Generar el cronograma de presentaciones y el Excel de calificación de panelistas (ver §9)

Todas son archivos HTML estáticos sin back-end propio. La persistencia (cuando aplica) se hace a través de servicios externos gratuitos/freemium. `index.html` y `programador.html` no escriben datos: solo leen/generan.

Existe además un **prototipo en pausa**, `generador-cohorte.html` (+ funciones serverless), para leer el calendario académico con IA y generar el cronograma/sitio de una cohorte. Se difirió por un límite del plan de Netlify y se retomará al migrar a Supabase — ver §10.

2. Arquitectura general



Flujo de datos resumido

1. El panelista abre `panelistas.html`, revisa proyectos y hace clic en "Quiero asistir".
2. Completa nombre y email → se envía a **Netlify Forms** (registro permanente) y a **EmailJS** (email de confirmación al panelista).
3. Inmediatamente aparece el **modal de logística**, donde declara transporte, fechas, dirección, hora de recogida, y si almuerza/desayuna en cada fecha aplicable.
4. Las preferencias logísticas se guardan en **JSONbin** con el email como clave.
5. El coordinador abre `admin.html`, ingresa la contraseña, y ve en tiempo real todas las respuestas sincronizadas desde JSONbin.
6. Desde admin puede editar manualmente la logística de cualquier panelista, enviar emails individuales (con o sin resumen de logística), generar un resumen por jornada, y exportarlo a Excel (CSV).

3. Servicios externos y credenciales

⚠ **Estas credenciales están embebidas en el HTML del lado del cliente.** Son aceptables para un prototipo interno, pero en una versión de producción deben moverse a variables de entorno o a un back-end.

3.1 Netlify

- **Plan:** Free (Starter)
- **Site name:** `navesfs`
- **URL de producción:** `https://navesfs.netlify.app`
- **Cuenta:** `jmvicaria@gmail.com`
- **Función usada:** Static hosting + Netlify Forms
- **Despliegue:** `netlify deploy --prod --dir .` desde la carpeta del proyecto
- **Forms dashboard:** `https://app.netlify.com/projects/navesfs/forms`

Netlify Forms captura automáticamente los formularios `data-netlify="true"` sin ningún back-end. Los envíos son visibles en el dashboard de Netlify. Incluye protección anti-spam (honeypot field `bot-field`).

3.2 EmailJS v4

- **Cuenta:** vinculada a `jmvicaria@gmail.com`
- **Public Key:** `LKIjrtq_CeLR_3Do6`
- **Service ID:** `service_6g8bgqk` (Gmail conectado)
- **Template ID:** `template_jow68si`
- **Variables del template:** `{{nombre}}`, `{{email}}`, `{{jornada}}`
- **Límite plan Free:** 200 emails/mes
- **Dashboard:** `https://dashboard.emailjs.com`

La librería de EmailJS v4 está **incrustada inline** en el HTML de `admin.html` (minificada) para evitar dependencia de CDN en producción. Se inicializa con:

```
emailjs.init('LKIjrtq_CeLR_3Do6');
```

El template `template_jow68si` debe tener configurado el campo `To Email` como `{{email}}` para que cada email vaya al destinatario correcto. El campo `{{jornada}}` contiene el texto de las jornadas confirmadas (y opcionalmente el resumen de logística cuando se usa "Enviar + logística").

3.3 JSONbin.io

Hay **dos bins** (misma cuenta y misma Master Key, documentos separados):

Bin	Bin ID	Usado por	Contenido
Logística	<code>6a2874d6f5f4af5e29d436aa</code>	<code>panelistas.html</code> , <code>admin.html</code>	Logística y confirmaciones de panelistas
Programación	<code>6a2f33e2da38895dfec095a4</code>	<code>programador.html</code>	Programación del evento (orden + horario)

- **Plan:** Free
- **Master Key (ambos bins):** `$2a$10$qBXsMcldU3zo7WzTazFhWe0rAvogSvSHw7bvSUBGWonD7zbLdj9hG`
- **Autenticación:** Header `X-Master-Key` (NO `X-Access-Key` — error frecuente)

JSONbin actúa como base de datos NoSQL de un solo documento JSON por bin.

Operaciones usadas:

```
GET /v3/b/{id}/latest → leer estado actual
PUT /v3/b/{id} → sobrescribir todo el documento
```

⚠ JSONbin no tiene control de concurrencia. Si dos usuarios guardan simultáneamente, el último en guardar gana. Aceptable para uso con un editor; con varios, refrescar antes de editar (el programador tiene el botón "🔄 Cargar nube" para esto).

4. Modelo de datos (JSONbin)

4.1 Bin de logística (`6a2874d6f5f4af5e29d436aa`)

El documento JSON tiene un único objeto raíz `logistica`, que es un diccionario con el **email del panelista como clave**:

```
{
  "logistica": {
    "email@panelista.com": {
      "transporte": "si" | "no" | null,
      "fechas_transporte": ["Viernes 12 de junio", "Sábado 13 de junio"] | null,
      "direccion": "Cra 15 #85-32, Bogotá" | null,
      "hora": "07:30" | null,
      "fechas_almuerzo": ["Viernes 12 de junio", "Viernes 19 de junio"] | null,
      "fechas_desayuno": ["Sábado 13 de junio"] | null,
      "enviado": true | false
    }
  }
}
```

Descripción de campos

Campo	Tipo	Descripción
<code>transporte</code>	<code>"si"</code> / <code>"no"</code> / <code>null</code>	Si el panelista solicita que lo recojan
<code>fechas_transporte</code>	<code>string[]</code> / <code>null</code>	Fechas específicas en que necesita recogida. <code>null</code> = sin definir
<code>direccion</code>	<code>string</code> / <code>null</code>	Dirección de recogida
<code>hora</code>	<code>string</code> (HH:MM) / <code>null</code>	Hora preferida de recogida
<code>fechas_almuerzo</code>	<code>string[]</code> / <code>null</code>	Fechas de Viernes en que almuerza. Array vacío <code>[]</code> = no almuerza ninguno
<code>fechas_desayuno</code>	<code>string[]</code> / <code>null</code>	Fechas de Sábado en que desayuna. Array vacío <code>[]</code> = no desayuna ninguno
<code>enviado</code>	<code>boolean</code>	<code>true</code> cuando el admin marcó que se envió email de confirmación

Valores posibles para fechas

```
"Viernes 12 de junio"
"Sábado 13 de junio"
"Viernes 19 de junio"
"Sábado 20 de junio"
```

Estos strings deben coincidir exactamente con los que se extraen del array `JORNADAS` mediante el regex `j.replace(/^.*- /, '').replace(/ .*$/, '').trim()`.

4.2 Bin de programación (`6a2f33e2da38895dfec095a4`)

El documento tiene un único objeto raíz `programacion`, que es el `snapshot()` del programador (`null` si nunca se ha guardado):

```
{
  "programacion": {
    "v": 1,
    "evento": "NAVES 2026",
    "tipo": "FS",
    "tiempos": { "expo":20, "trans":5, "foto":10, "cierre":20 },
    "dias": [
      { "fecha":"Viernes 12 de junio", "inicio":"13:50", "foto":true, "intro":20, "n":10,
        "interr":[ {"tipo":"break","min":30,"auto":true} ] }
    ],
    "orden": [ {"proyecto":"Bevo","autores":"...","sector":"RRHH / Bienestar"} ],
    "solicitudes": [ {"proyecto":"...","tipo":"temprano","detalle":"","nota":""} ]
  }
}
```

El **cronograma** calculado (cada proyecto con día, slot y hora) **no** se persiste aquí: se recalcula al cargar y solo se incluye en el JSON exportado (**exportarJSON** , ver §9.6b), que es el insumo para regenerar **index.html** . Ver §9.4 para el detalle de cada campo.

5. naves_generator.py — Script generador de index.html

5.1 Propósito

Script Python que automatiza la creación de **index.html** leyendo los archivos de OneDrive (logos, business plans, one pagers), extrayendo metadatos de los nombres de archivo, y combinando todo con resúmenes y posts de LinkedIn pre-escritos.

Ubicación: /Users/juanmanuel/Dropbox/INALDE/NAVES/NAVES_Agentes/naves_generator.py

Output: /Users/juanmanuel/Desktop/NAVES_2026_Web/index.html (+ copia de logos y PDFs a **logos/** y **pdfs/**)

Ejecutar:

```
cd /Users/juanmanuel/Dropbox/INALDE/NAVES/NAVES_Agentes
python3 naves_generator.py
```

Requiere Python 3.8+ con **pdfplumber** instalado:

```
pip install pdfplumber
```

5.2 Carpetas de entrada (OneDrive, solo lectura)

```
OneDrive-INALDEBusinessSchool-UniversidaddeLaSabana/
├── Logotipo MBA FS 24-26/      → logos JPG/PNG de cada proyecto
├── Business Plan MBA FS 24-26/ → business plans en PDF
├── Resumen MBA FS 24-26/      → one pagers en PDF, PNG o JPG
```

Las carpetas de entrada **nunca se modifican**. El script solo lee de ellas.

Ruta base en macOS:

```
ONEDRIVE = Path("/Users/juanmanuel/Library/CloudStorage/OneDrive-INALDEBusinessSchool-Uni
```

5.3 Convención de nombres de archivo

Los archivos siguen el patrón:

```
[Autor1]-[Autor2]-[Autor3]_[TIPO]_2026_MBA_FS_[PROYECTO].[ext]
```

Ejemplos reales:

```
Héctor_Rodrigo_Arias_Cueca-Cesar_Julián_Pérez_Garavito_logo_2026_MBA_FS_AKOS.jpg
Edwin_Muñoz_Aristizabal_BUSINESS_PLAN_2026_MBA_FS_BrokerLLM.pdf
Sebastian_Lopez-John_Segura-Claudia_Estevez_logo_2026_MBA_FS_LaEtapaCafe.jpg
```

- **Autores:** separados por guion (-); cada autor tiene nombre y apellidos separados por _
- **Tipo:** logo , Logo , BUSINESS PLAN , RESUMEN , ONE PAGER
- **Proyecto:** al final, antes de la extensión. Puede ser camelCase o con espacios.

5.4 Pipeline de procesamiento (pasos en orden)

Paso 1 — normalize(s)

Elimina acentos, pasa a minúsculas, elimina todo lo que no sea alfanumérico. Usado para comparar nombres de proyecto sin sensibilidad a tildes/mayúsculas/guiones.

```
def normalize(s):
    s = unicodedata.normalize('NFD', s)
    s = ''.join(c for c in s if unicodedata.category(c) != 'Mn')
    return re.sub(r'^[a-z0-9]', '', s.lower())
```

Paso 2 — extract_project(fname) y extract_authors(fname)

extract_project extrae el nombre del proyecto al final del nombre de archivo:

```
m = re.search(r'MBA_F[SD]S?_(.+?)(?:\.\w+)+$', fname, re.IGNORECASE)
```

Después aplica **NAME_CORRECTIONS** para unificar variantes (clic07 → CLIC , fioya → FioYa).

extract_authors extrae los autores del prefijo del nombre:

```
m = re.match(r'^(.+?)(?:_RESUMEN|_ONE PAGER|_BUSINESS|_logo|_Logo)', fname, re.IGNORECASE)
# Luego split por '-' para obtener cada autor
```

Paso 3 — `scan_projects()`

Escanea las 3 carpetas y construye un diccionario:

```
projects = {
  "AKOS": {
    "logo": "Héctor Rodrigo Arias Cueca-..._logo_2026_MBA_FS_AKOS.jpg",
    "bp": "..._BUSINESS PLAN_2026_MBA_FS_AKOS.pdf",
    "resumen": "..._RESUMEN_2026_MBA_FS_AKOS.pdf",
    "authors": ["Héctor Rodrigo Arias Cueca", "Cesar Julián Pérez Garavito"],
    "name": "AKOS"
  },
  ...
}
```

La clave de matching es el nombre normalizado. Si el mismo proyecto aparece en varias carpetas, se fusiona en un solo registro.

Paso 4 — `read_pdf_text(folder, fname, max_chars=800)`

Lee las primeras 4 páginas de un PDF usando `pdfplumber` y retorna hasta 800 caracteres de texto plano. **Limitación clave:** `pdfplumber.extract_text()` solo recupera texto vectorial; con one pagers **diseñados como imagen** (exportados desde Canva/Figma/InDesign con el texto rasterizado) devuelve cadena vacía. Cerca de la mitad de los one pagers de la cohorte 2024-2026 son de este tipo. Esta función está en desuso y, además, **no es suficiente** como única fuente — ver el incidente documentado en el Paso 5.

Paso 5 — `SUMMARIES_DATA` (resúmenes precargados)

Lista de 34 diccionarios `{proyecto, resumen, linkedin}`:

```
SUMMARIES_DATA = [
  {"proyecto": "AKOS",
   "resumen": "FinTech colombiana que financia medicamentos...",
   "linkedin": "Héctor Rodrigo Arias Cueca y Cesar Julián... #SoyINALDE ..."},
  ...
]
```

Historia: En la primera versión del script, se llamaba a la Claude API para generar estos textos automáticamente desde el PDF de cada proyecto. Para la versión final, los resúmenes y posts fueron hardcodeados en `SUMMARIES_DATA` y `generate_summaries_and_posts()` pasó a retornarlos sin llamar a ninguna API.

⚠ Incidente del 16 jun 2026 — descripciones inventadas (causa raíz)

Al hardcodear los textos se perdió el vínculo con la fuente. Varios resúmenes/posts quedaron **inventados o de un negocio equivocado**, porque cuando se redactaron a mano el one pager era una **imagen** (texto no extraíble por `pdfplumber`) y se rellenó "a ciegas". Casos detectados y corregidos en producción ese día (10 proyectos): **Bio Value** (decía "ingredientes para industria

alimentaria" cuando es **hilados/fibras textiles** — negocio equivocado), **T-HEALTH** ("salud corporativa de colaboradores" cuando es **monitoreo predictivo para EPS/aseguradoras**), **Mony** ("fintech genérica de crédito" cuando es **hábitos financieros para madres cabeza de familia vía WhatsApp, pagado por el empleador**), **PLICS** ("toma de decisiones complejas" cuando es **e-Procurement B2B**), **CLIC** ("computación cuántica para procesamiento empresarial" cuando es **hub de I+D deep tech / time-to-discovery con talento PhD**), **Avia Style** ("dotar viviendas para rentar" cuando es **dotación llave en mano / Furniture as a Service del pool de rentas de AVIA Suites**), más ajustes de fidelidad en Ready2, Zafiro, La Etapa Café y Plug&GoEV. La corrección se hizo directo sobre `index.html` / `panelistas.html` (no sobre el script), por lo que **el `SUMMARIES_DATA` del script sigue conteniendo los textos viejos**: regenerar con el script actual reintroduciría los errores.

✅ Requisito de diseño para futuras versiones (obligatorio)

Decisión del 16 jun 2026: **los resúmenes y los posts de LinkedIn deben generarse SIEMPRE con IA leyendo la fuente real de cada proyecto, sea cual sea su formato**. Nunca volver a hardcodear ni redactar "a ciegas". Reglas:

1. **La fuente manda.** El texto se destila del one pager (o del business plan si hace falta). Si el one pager es **imagen** (PDF rasterizado, PNG, JPG), se debe leer con un modelo de **visión** (renderizar el PDF a imagen con `PyMuPDF` / `pdf2image` y pasarlo a Claude con visión) — nunca asumir el contenido a partir del nombre o del sector.
2. **Modelo con visión, sin límite de tiempo.** Usar `claude-opus-4-8` (o el mejor disponible) por la calidad de lectura de imágenes densas (one paggers con datos pequeños). En Supabase, vía Edge Function (ver §17); fuera de Netlify de bajo plan, que no soportó este tipo de carga (ver §10).
3. **Caché por hash de la fuente.** Guardar `{proyecto → {resumen, linkedin, source_sha256, generado_en, aprobado}}`. Solo se vuelve a llamar a la IA si el one pager cambió (hash distinto) o el proyecto es nuevo. Así no se paga por reproceso ni se machacan textos ya aprobados.
4. **Nada se publica sin sustento.** Si un proyecto no tiene one pager legible y no hay forma de generarlo con IA, el build **falla con error explícito** en vez de inventar. Prohibido el fallback a texto genérico.
5. **Revisión humana ligera.** Lo generado por IA queda marcado `aprobado: false` hasta que el coordinador lo confirme; lo aprobado se preserva entre corridas.
6. **El post de LinkedIn** se arma con los autores reales (de `extract_authors`) + la descripción fiel + los hashtags estándar (`HT`). La frase de cierre puede ser de la IA pero debe desprenderse del contenido del one pager, no de un cliché.

Implementación recomendada: en la migración a Supabase, una Edge Function `generar-descripciones` que recibe el one pager (de Storage), lo lee con Opus (visión incluida) y devuelve `{resumen, linkedin}` para revisión. Es el mismo patrón que la lectura del calendario (§10/§17), reutilizable.

Hashtags estándar (constante `HT`):

```
HT = "#SoyINALDE #NavesINALDE #ExecutiveMBA #Líder #INALDE #Liderazgo #MBA #EMBA #NAVES"
```

Paso 6 — `prepare_output(projects)`

Copia archivos de OneDrive a la carpeta de salida con nombres seguros:

- Logo → `logos/{safe_name}.{ext}` (ej: `logos/AKOS.jpg`)
- One pager → `pdfs/{safe_name}_resumen.{ext}` (ej: `pdfs/AKOS_resumen.pdf`)
- Business plan → `pdfs/{safe_name}_bp.pdf` (no se usa en el HTML público, solo se copia)

Nombre seguro: `safe = re.sub(r'^\w[-]', '_', proj)` (reemplaza todo excepto letras, números, guion y underscore).

One pagers pueden ser `.pdf`, `.png`, `.jpg` o `.jpeg` — el script acepta todos.

Paso 7 — `generate_html(projects, summaries_data)`

Genera el HTML completo:

1. Construye `summary_map` (dict `normalize(proyecto) → {resumen, linkedin}`)
2. Ordena proyectos por sector, luego por nombre (para la versión generada por script)
3. Para cada proyecto construye una fila `<tr>` con todos los `data-*` attributes
4. Inserta filas de encabezado de sector (`<tr class="sector-header">`) cuando cambia el sector
5. Genera contadores (`total`, `completos`) para las stats del hero
6. Retorna el HTML completo como string

DIFERENCIA CRÍTICA: La versión que genera el script agrupa por sector. La versión desplegada en producción (`index.html` actual) está reorganizada manualmente para seguir el orden del calendario de presentaciones (día + slot). Esta reorganización **no está automatizada** en el script. Al regenerar con el script, el HTML quedará en orden por sector y deberá reorganizarse a mano, o modificar el script para que acepte un mapa de horarios.

5.5 Constantes de configuración del script

Constante	Descripción
<code>SECTORES</code>	Dict <code>{nombre_proyecto → sector}</code> para 35 proyectos. Hay que actualizarlo si entran proyectos nuevos.
<code>SECTOR_COLORS</code>	Dict <code>{sector → color_hex}</code> — 8 sectores con colores distintos.
<code>NAME_CORRECTIONS</code>	Dict <code>{normalize(variante) → nombre_canónico}</code> para nombres que se escriben diferente en los archivos. Ej: <code>"clíc07" → "CLIC"</code> .
<code>CONFIDENCIALES</code>	Set de nombres de proyectos sin descargas. Actualmente solo <code>{"CDO Alianza"}</code> .

5.6 Cómo agregar un proyecto nuevo

1. Subir logo, one pager y business plan a las carpetas de OneDrive correspondientes, siguiendo la convención de nombres.

2. Agregar `"NombreProyecto": "Sector"` al dict `SECTORES`.
3. Si el nombre del proyecto tiene variantes en los archivos, agregar a `NAME_CORRECTIONS`.
4. Escribir resumen y post LinkedIn y agregar un dict a `SUMMARIES_DATA`.
5. Ejecutar `python3 naves_generator.py`.
6. Reorganizar las filas del HTML generado para ubicar el proyecto en el slot correcto del calendario.
7. Desplegar: `netlify deploy --prod --dir /Users/juanmanuel/Desktop/NAVES_2026_Web`.

5.7 Salida del script en consola

```
🚀 NAVES 2026 – Generador HTML
=====
📁 Paso 1: Escaneando carpetas...
→ 34 proyectos encontrados

📄 Paso 2: Preparando archivos de salida...
→ Archivos copiados a /Users/juanmanuel/Desktop/NAVES_2026_Web

🤖 Paso 3: Generando resúmenes con Claude API...
Usando resúmenes generados desde los PDFs...
→ 34 resúmenes generados

🌐 Paso 4: Generando HTML...
→ HTML guardado en: /Users/juanmanuel/Desktop/NAVES_2026_Web/index.html

📊 Estado de entregas:
[✓✓✓] AKOS
[✓✓✓] AMIGO
[✓x✓] Grey2Blue      ← sin business plan
...

✅ 31/34 proyectos completos (de 45 esperados)
```

El conteo "de 45 esperados" refleja el objetivo original del programa; al final fueron 34 proyectos completos.

5.8 Limitaciones y recomendaciones

Para la versión profesional:

- **Automatizar el orden por horario:** Crear un CSV o dict `SCHEDULE = {"AKOS": {"dia": "Viernes 12 jun", "slot": 3, "hora_inicio": "14:55", "hora_fin": "15:15"}}` y que `generate_html()` lo use para ordenar y generar los encabezados de día.
- **Separar datos del template:** La lógica de negocio (qué proyectos existen, sus textos) está mezclada con el template HTML. Separar en: (a) script que genera `projects.json`; (b) template HTML que consume el JSON.
- **Subir directamente a Netlify:** Actualmente hay que ejecutar el script, verificar el HTML manualmente, y luego correr `netlify deploy`. Se podría automatizar con un workflow de GitHub Actions que se ejecute al hacer push al repo.

- **Control de versiones para los textos:** `SUMMARIES_DATA` está hardcodeado en el script Python. Moverlo a un CSV o Airtable permitiría que alguien sin conocimientos de Python edite los textos.

6. index.html — Base de datos de proyectos

6.1 Propósito y audiencia

`index.html` es la base de datos interna de proyectos del NAVES 2026, destinada a **usuarios internos de INALDE**. No es para los panelistas (ellos tienen `panelistas.html`) ni para los estudiantes. Tiene tres perfiles de uso con necesidades distintas:

Perfil	Qué necesita de esta página
Comunicaciones	Posts de LinkedIn listos para copiar y pegar; logos descargables en alta resolución para publicaciones en redes sociales
Publicaciones	One pagers descargables para armar materiales impresos o digitales del programa
Asistentes de programa	Calendario completo de presentaciones: qué proyecto presenta en qué fecha, jornada, slot y hora; Excel de programación para coordinar logística del evento

URL: `https://navesfs.netlify.app/index.html` (o simplemente `https://navesfs.netlify.app/`)

Acceso: Sin contraseña en el prototipo actual. En la versión profesional debería requerir autenticación con cuenta INALDE — los one pagers y modelos financieros son documentos estratégicos. Los posts de LinkedIn y resúmenes pueden mantenerse sin restricción.

6.2 Funcionalidades actuales

Función	Perfil que la usa	Descripción
Filtro por día	Asistentes de programa	Botones: Todos / Vie 12 jun / Sáb 13 jun / Vie 19 jun / Sáb 20 jun
Búsqueda libre	Todos	Filtra en tiempo real por proyecto, autor o sector
Slot de presentación	Asistentes de programa	Cada fila muestra el número de slot y su horario exacto (ej. "1 / 13:50–14:10")
Badge de sector	Todos	Color codificado, 8 sectores distintos
One Pager	Publicaciones	Link "Ver One Pager →" abre en pestaña nueva; botón "↓ One Pager" descarga el archivo
Post LinkedIn	Comunicaciones	Texto completo + botón "Copiar" con feedback visual "✓ Copiado" por 2 segundos
Logo descargable	Comunicaciones	Botón "↓ Logo" descarga el JPG original
Proyecto confidencial	—	CDO Alianza aparece como "🔒 Confidencial" sin botones de descarga
Exportar Excel	Comunicaciones / Asistentes	Botón verde "↓ Descargar Excel" — genera NAVES_2026_Proyectos.xlsx con SheetJS

6.2b Funcionalidades pendientes para versiones futuras

Función	Perfil	Descripción
Excel de programación	Asistentes de programa	Exportar el horario completo del evento: una fila por slot con fecha, jornada, número de slot, hora inicio, hora fin, proyecto y autores. Ver especificación completa en §6.8.
Autenticación	Todos	Login con cuenta INALDE antes de acceder a one pagers y descargas
Filtro por sector	Todos	Actualmente el script genera encabezados de sector pero el sitio desplegado no tiene ese filtro activo — solo filtra por día

6.3 Estructura de la tabla

Columnas:

- Slot** — número de presentación + horario (`<strong>N</strong><br><span class="slot-time">HH:MM–HH:MM</span>`)
- Logo** — miniatura de la imagen del logo (`<img class="logo-thumb">` o `<div class="logo-placeholder">`)
- Proyecto** — nombre en negrita
- Autores** — separados por coma
- Sector** — badge coloreado
- One Pager** — resumen de 250 caracteres + link "Ver One Pager →"

7. **Post LinkedIn** — texto completo + botón Copiar

8. **Descargas** — botones ↓ Logo y ↓ One Pager

Datos en `data-*` (usados por filtros y exportación Excel):

```
<tr data-day="Viernes 12 jun"
    data-sector="RRHH / Bienestar"
    data-proj="Bevo"
    data-authors="Silvio Andrés Terán Calvache, ..."
    data-resumen="Plataforma de bienestar corporativo..."
    data-linkedin="Silvio Andrés... #SoyINALDE ..."
    data-logo="logos/Bevo.jpg"
    data-resumen-url="pdfs/Bevo_resumen.pdf">
```

Separadores de día: filas `<tr class="day-header" data-day="Viernes 12 jun">` con fondo negro que encabezan cada grupo de presentaciones de ese día.

6.4 Orden de los proyectos

Los proyectos están ordenados por **día de presentación + slot**, que es el orden cronológico del evento. Este es el único criterio de ordenamiento que tiene sentido para los usuarios de este sitio.

Día	Slot 1 empieza	Intervalo entre slots
Viernes 12 jun	13:50	20 min por proyecto + 5 min pausa
Sábado 13 jun	07:20	ídem
Viernes 19 jun	14:00	ídem
Sábado 20 jun	08:55	ídem

Sobre el criterio de sector: el script `naves_generator.py` genera el HTML agrupando proyectos por sector porque así fue diseñado originalmente para facilitar la generación automática. Sin embargo, **agrupar presentaciones por sector no es un criterio de programación del evento** — los proyectos se asignan a slots por razones logísticas (disponibilidad de panelistas, número de proyectos por jornada), no por afinidad sectorial. El orden del HTML desplegado fue reorganizado manualmente para reflejar el horario real.

IMPORTANTE para el próximo desarrollador: El orden por día/slot del `index.html` actual **fue ingresado manualmente**. El script `naves_generator.py` sigue generando las filas por sector. Al regenerar con el script, el orden quedará incorrecto y deberá reorganizarse — o debe modificarse el script para recibir un mapa `{proyecto → {dia, slot, hora_inicio, hora_fin}}` y generar el HTML ya en orden cronológico. Ver §5.8 y §16.10.

6.5 Filtros JavaScript

```
let activeDay = 'all'; // Filtro de día activo

function setDay(day, btn) {
  activeDay = day;
  document.querySelectorAll('.filter-btn').forEach(b => b.classList.remove('active'));
  btn.classList.add('active');
  filterTable();
}

function filterTable() {
  const q = document.getElementById('search-input').value.toLowerCase();
  let visible = 0;
  document.querySelectorAll('#table-body tr:not(.day-header)').forEach(row => {
    const text = row.textContent.toLowerCase();
    const day = row.dataset.day || '';
    const show = (!q || text.includes(q)) && (activeDay === 'all' || day === activeDay);
    row.style.display = show ? '' : 'none';
    if (show) visible++;
  });
  // Ocultar separadores de día si no tienen filas visibles
  document.querySelectorAll('.day-header').forEach(dh => {
    let next = dh.nextElementSibling, hasVisible = false;
    while (next && !next.classList.contains('day-header')) {
      if (next.style.display !== 'none') hasVisible = true;
      next = next.nextElementSibling;
    }
    dh.style.display = hasVisible ? '' : 'none';
  });
  document.getElementById('count-label').textContent =
    `Mostrando ${visible} proyecto${visible !== 1 ? 's' : ''}`;
}
```

6.6 Exportación a Excel (SheetJS)

```
function downloadExcel() {
  const wb = XLSX.utils.book_new();
  const wsData = [['Proyecto', 'Autores', 'Sector', 'One Pager', 'Post LinkedIn', 'Logo']];
  const dataRows = document.querySelectorAll('#table-body tr[data-proj]');
  const base = window.location.origin + window.location.pathname.replace(/\/[^\/]*$/, '/');

  dataRows.forEach(row => {
    wsData.push([
      row.dataset.proj,
      row.dataset.authors,
      row.dataset.sector,
      row.dataset.resumen,
      row.dataset.linkedin.replace(/&/g, '&').replace(/"/g, ''),
      row.dataset.logo ? base + row.dataset.logo : ''
    ]);
  });

  const ws = XLSX.utils.aoa_to_sheet(wsData);
  ws['!cols'] = [{wch:20}, {wch:40}, {wch:22}, {wch:62}, {wch:95}, {wch:55}];

  // Agregar hipervínculos a las celdas de One Pager (col D) y Logo (col F)
  dataRows.forEach((row, i) => {
    const excelRow = i + 2;
    const resUrl = row.dataset.resumenUrl ? base + row.dataset.resumenUrl : '';
    const logoUrl = row.dataset.logo ? base + row.dataset.logo : '';
    if (ws['D' + excelRow] && resUrl)
      ws['D' + excelRow] = { v: ws['D' + excelRow].v, t: 's', l: { Target: resUrl } };
    if (ws['F' + excelRow] && logoUrl)
      ws['F' + excelRow] = { v: logoUrl, t: 's', l: { Target: logoUrl } };
  });

  XLSX.utils.book_append_sheet(wb, ws, 'Proyectos NAVES 2026');
  XLSX.writeFile(wb, 'NAVES_2026_Proyectos.xlsx');
}
```

Librería: SheetJS (XLSX.js) versión 0.20.3, cargada desde CDN:

```
<script src="https://cdn.sheetjs.com/xlsx-0.20.3/package/dist/xlsx.full.min.js"></script>
```

El Excel exportado tiene hipervínculos en la columna D (One Pager) y F (Logo) que apuntan a las URLs absolutas del sitio Netlify.

6.7 Copiar post LinkedIn

```
function copyText(btn, text) {
  navigator.clipboard.writeText(
    text.replace(/&/g, '&').replace(/</g, '<').replace(/>/g, '>').replace(/"/g, '"')
  ).then(() => {
    btn.textContent = '✓ Copiado';
    btn.classList.add('copied');
    setTimeout(() => { btn.textContent = 'Copiar'; btn.classList.remove('copied'); }, 200);
  });
}
```

Los textos de LinkedIn están escapados con HTML entities en los atributos `data-linkedin`. La función los deserializa antes de copiar al portapapeles.

6.8 Excel de programación (funcionalidad pendiente para asistentes de programa)

El botón "↓ Descargar Excel" actual genera un archivo orientado a comunicaciones (proyecto, autores, sector, resumen, post LinkedIn, logo). Las **asistentes de programa** necesitan un Excel diferente: el horario de presentaciones del evento, que les permita coordinar la logística del día.

Especificación del Excel de programación:

Nombre del archivo: `NAVES_2026_Programacion.xlsx`

Columnas (una fila por slot de presentación):

Col	Nombre	Ejemplo	Notas
A	Fecha	Viernes 12 de junio de 2026	Fecha completa del día
B	Jornada	Jornada 1	Número de jornada en el total del evento
C	Slot	3	Posición dentro de la jornada
D	Hora inicio	14:40	HH:MM
E	Hora fin	15:00	HH:MM
F	Proyecto	AKOS	Nombre del proyecto
G	Autores	Héctor Rodrigo Arias Cueva, Cesar Julián Pérez Garavito	Separados por coma
H	Sector	FinTech / Financiero	
I	One Pager	https://navesfs.netlify.app/pdfs/AKOS_resumen.pdf	Hipervínculo al PDF

Formato adicional sugerido:

- Una fila separadora (fondo gris oscuro) entre cada día de presentación
- Columnas D y E con formato de hora (no texto)

- Ancho de columna ajustado para impresión A4 horizontal
- Una segunda hoja "Resumen" con el conteo de slots por jornada y el número total de proyectos

Implementación: En el `index.html` actual, toda la información de fecha y slot ya está en los atributos `data-day` y el texto visible de la columna Slot. Falta agregar `data-slot`, `data-hora-inicio` y `data-hora-fin` como atributos en cada `<tr>` para que `downloadExcel()` pueda leerlos. Alternativamente, en la versión profesional este Excel se genera en el servidor directamente desde la base de datos de `slot_presentacion` (ver §16.4).

6.9 Estructura de archivos de soporte

```
NAVES_2026_Web/
├── index.html
├── logos/
│   ├── Bevo.jpg
│   ├── SATORI.jpg
│   ├── AKOS.jpg
│   └── ... (34 archivos, nombres safe = re.sub(r'^\w\-', '_', nombre_proyecto))
└── pdfs/
    ├── Bevo_resumen.pdf
    ├── SATORI_resumen.pdf
    └── ... (one pagers; algunos son .png o .jpg en lugar de .pdf)
```

Los nombres de archivo en `logos/` y `pdfs/` son el nombre canónico del proyecto con caracteres especiales reemplazados por `_` (usando `re.sub(r'^\w\-', '_', proj)`). Ej: `La_Etapa_Caf_` para "La Etapa Café".

6.9 Relación con panelistas.html

`index.html` y `panelistas.html` **son independientes**. `panelistas.html` embebe los logos directamente como Base64 (para evitar dependencias de ruta), mientras que `index.html` los carga como archivos externos desde `logos/`.

7. panelistas.html — Documentación detallada

7.1 Propósito

Página pública enviada a los 12 panelistas por email o WhatsApp. Permite:

- Revisar los 34 proyectos organizados por jornada
- Descargar el One Pager (PDF) de cada emprendimiento
- Confirmar asistencia a jornadas específicas o a todas
- Declarar preferencias de logística (transporte y comidas)

7.2 URL

```
https://navesfs.netlify.app/panelistas.html
```

7.3 Estructura de la página

<header>	Logo NAVES + badge "Panel de Evaluación"
<hero>	Título, descripción, instrucciones
<todas-banner>	Banner "Confirmar todas las jornadas" (arriba del todo)
<contenido>	7 jornada-cards, una por jornada
<modal-logistica>	Modal de preferencias logísticas (se abre tras confirmar)

7.4 Jornadas y proyectos hardcodedos

Las 7 jornadas están hardcodedas en el HTML. Cada jornada tiene:

- Número (Jornada 1–7)
- Fecha y hora (ej: "Viernes 12 de junio · 13:50 – 15:50")
- 4–5 proyectos con: slot horario, logo, nombre, autores, resumen de 2 líneas, botón "↓ One Pager" (PDF)

Los PDFs están en la carpeta `pdfs/` y los logos en `logos/`. El botón One Pager abre el PDF en nueva pestaña.

Proyectos por jornada:

Jornada	Fecha	Hora	Proyectos
1	Viernes 12 jun	13:50–15:50	Bevo, SATORI, VERIMED, Chargehub, Plug&GoEV
2	Viernes 12 jun	16:30–18:30	(5 proyectos)
3	Sábado 13 jun	07:20–09:20	(5 proyectos)
4	Sábado 13 jun	10:00–12:00	(5 proyectos)
5	Viernes 19 jun	14:00–16:00	(5 proyectos)
6	Viernes 19 jun	16:40–18:40	(4 proyectos)
7	Sábado 20 jun	08:55–10:30	(5 proyectos)

7.5 Flujo de confirmación individual

1. El panelista hace clic en "👉 **Quiero asistir a esta jornada**" al final de la jornada-card.
2. El botón llama a `showForm(num, label)` que muestra/oculta el formulario.
3. El formulario pide **Nombre** y **Email** (ambos requeridos).
4. Al enviar, `handleSubmit(event, num)` hace dos cosas en paralelo:
 - POST a Netlify Forms (registro de la asistencia)
 - Llama a `enviarConfirmacion(nombre, email, jornada)` → EmailJS
5. Guarda en `_pendingSuccessInfo = { type: 'individual', num: num }`.
6. Llama a `mostrarLogistica(email, jornada)` → abre el modal.

```
// Llamada central de confirmación individual
handleSubmit(event, num)
  → fetch('/', POST) // Netlify Forms
  → enviarConfirmacion(nombre, email, jornada) // EmailJS
  → mostrarLogistica(email, jornada) // Modal logística
```

7.6 Flujo de confirmación "Todas las jornadas"

1. El panelista hace clic en "👉 **Quiero asistir a todas las jornadas**" en el banner superior.
2. `showTodasForm()` muestra el formulario dentro del banner.
3. Al enviar, `handleTodasSubmit(event)` hace:
 - POST a Netlify Forms con `jornada = "TODAS las jornadas – 12, 13, 19 y 20 de junio"`
 - EmailJS con el mismo texto de jornada
 - `_pendingSuccessInfo = { type: 'todas' }`
 - `mostrarLogistica(email, jornada)`

7.7 enviarConfirmacion — EmailJS

```
function enviarConfirmacion(nombre, email, jornada) {
  emailjs.send('service_6g8bgqk', 'template_jow68si', {
    nombre: nombre,
    email: email,
    jornada: jornada
  });
}
```

El email va al panelista confirmando qué jornadas asistirá. Los errores de EmailJS se ignoran silenciosamente (no bloquean el flujo).

7.8 Modal de logística

El modal (`#modal-logistica`) se abre **automáticamente** tras cada confirmación. No puede saltarse fácilmente (solo con el botón "Omitir por ahora" o cerrando la X). Si se cierra sin guardar, se llama igualmente a `mostrarExito()` para mostrar el mensaje de confirmación.

Variables de estado del modal

```
var _logEmail    = ''; // email del panelista que acaba de confirmar
var _logJornada  = ''; // jornada que confirmó (texto completo)
var _pendingSuccessInfo = null; // { type: 'individual'|'todas', num? }
```

Secciones del modal

1. Transporte

- Radios: "Sí, necesito transporte" / "No, iré por mi cuenta"
- Al seleccionar "Sí", se despliega `#l-transport-details` con:
- Para jornada individual: etiqueta informativa con la fecha (no editable)

- Para "todas": checkboxes de las 4 fechas (Vie 12, Sáb 13, Vie 19, Sáb 20), todas marcadas por defecto
- Campo texto: dirección de recogida
- Campo time: hora preferida

2. Almuerzo (solo visible si el panelista tiene algún Viernes)

- Checkboxes por cada Viernes aplicable:
- `#l-alm-cb-v12` → "Viernes 12 de junio" (visible si la jornada incluye el 12)
- `#l-alm-cb-v19` → "Viernes 19 de junio" (visible si la jornada incluye el 19)
- Marcadas por defecto si el panelista tiene esa fecha

3. Desayuno (solo visible si el panelista tiene algún Sábado)

- Checkboxes por cada Sábado aplicable:
- `#l-des-cb-s13` → "Sábado 13 de junio"
- `#l-des-cb-s20` → "Sábado 20 de junio"

Lógica de visibilidad en mostrarLogistica

```
function mostrarLogistica(email, jornada) {
  _logEmail    = email;
  _logJornada  = jornada;
  // ...reset de campos...

  var j = jornada.toLowerCase();
  var esTodas = j.indexOf('todas') !== -1;
  var tieneV12 = j.indexOf('12') !== -1 || esTodas;
  var tieneV19 = j.indexOf('19') !== -1 || esTodas;
  var tieneS13 = j.indexOf('13') !== -1 || esTodas;
  var tieneS20 = j.indexOf('20') !== -1 || esTodas;

  // Transporte: individual muestra texto, todas muestra checkboxes
  // Almuerzo: visible si tieneV12 || tieneV19
  // Desayuno: visible si tieneS13 || tieneS20
}
```

guardarLogisticaPanelista — guardado en JSONbin

```
async function guardarLogisticaPanelista() {
  // 1. Leer valores del modal
  var transporte = ...;
  var direccion = ...;
  var hora = ...;
  var fechasTransporte = []; // recoger checkboxes marcados
  var fechasAlmuerzo = []; // recoger checkboxes marcados
  var fechasDesayuno = []; // recoger checkboxes marcados

  // 2. GET /latest para no sobrescribir datos de otros panelistas
  var r = await fetch(LOG_BIN_URL + '/latest', { headers: { 'X-Master-Key': LOG_BIN_KEY } });
  var j = await r.json();
  var logistica = j.record.logistica || {};

  // 3. Merge con datos existentes del mismo email
  logistica[_logEmail] = Object.assign({}, logistica[_logEmail] || {}, { ... });

  // 4. PUT con el documento completo actualizado
  await fetch(LOG_BIN_URL, { method: 'PUT', ... body: JSON.stringify({logistica}) });

  // 5. Cerrar modal y mostrar éxito
  mostrarExito();
}
```

⚠ **Punto crítico de concurrencia:** El patrón GET-then-PUT no es atómico. Si dos panelistas confirman exactamente al mismo tiempo, uno puede sobrescribir al otro. En la práctica con 12 panelistas esto es improbable, pero en una versión de producción se debe usar una API que soporte operaciones atómicas (Firestore, Supabase, etc.).

mostrarExito

```
function mostrarExito() {
  var info = _pendingSuccessInfo;
  _pendingSuccessInfo = null;
  if (info.type === 'individual') {
    // Muestra #success-{num}, deshabilita botón, cambia texto a "✅ Asistencia confirmada"
  } else if (info.type === 'todas') {
    // Muestra #todas-success, deshabilita #btn-todas
  }
}
```

7.9 Credenciales en panelistas.html

```
var LOG_BIN_URL = 'https://api.jsonbin.io/v3/b/6a2874d6f5f4af5e29d436aa';
var LOG_BIN_KEY = '$2a$10$qBXsMcldU3zo7WzTazFhWe0rAvogSvSHw7bvSUBGWonD7zbLdj9hG';
```

EmailJS se inicializa en `enviarConfirmacion` :

```
emailjs.send('service_6g8bgqk', 'template_jow68si', { nombre, email, jornada });
```

7.10 Assets necesarios

```
/logos/           Logos de los 34 emprendimientos (JPG/PNG/JPEG)
/pdfs/            One Pagers de los 34 emprendimientos (PDF)
/og-panelistas.jpg Imagen Open Graph para WhatsApp (1200x630px)
```

8. admin.html — Documentación detallada

8.1 Propósito

Herramienta interna para el coordinador de NAVES. Permite:

- Ver el estado logístico de todos los panelistas en tiempo real
- Editar manualmente la logística de cualquier panelista
- Enviar emails de confirmación (con o sin resumen de logística)
- Ver el resumen por jornada (texto + descarga Excel/CSV)
- Ver estadísticas agregadas

8.2 URL

```
https://navesfs.netlify.app/admin.html
```

8.3 Autenticación

Protección por contraseña simple del lado del cliente:

```
const PWD = 'Mba2026';

function checkPwd() {
  if (input.value === PWD) {
    // Ocultar login-screen, mostrar admin-content, llamar cargarDatos()
  } else {
    // Mostrar login-error
  }
}
```

⚠ Esta autenticación **no es segura** para datos sensibles. Es adecuada para un prototipo interno. En producción debe reemplazarse por autenticación real (OAuth, JWT, etc.).

La pantalla de login muestra el logo NAVES sobre fondo negro con un campo de contraseña. Al presionar Enter o hacer clic en el botón se valida.

8.4 Constantes de configuración

```
const PWD      = 'Mba2026';           // Contraseña del admin
const SVC      = 'service_6g8bgqk';   // EmailJS Service ID
const TPL      = 'template_jow68si';  // EmailJS Template ID
const BIN_ID   = '6a2874d6f5f4af5e29d436aa';
const BIN_KEY  = '$2a$10$qBXsMcldU3zo7WzTazFhWe0rAvogSvSHw7bvSUBGWonD7zbLdj9hG';
const BIN_URL  = `https://api.jsonbin.io/v3/b/${BIN_ID}`;

const TODAS_LABEL = 'TODAS las jornadas – Viernes 12, Sábado 13, Viernes 19 y Sábado 20 d
const JORNADAS = [
  'Jornada 1 – Viernes 12 de junio · 13:50 – 15:50',
  'Jornada 2 – Viernes 12 de junio · 16:30 – 18:30',
  'Jornada 3 – Sábado 13 de junio · 07:20 – 09:20',
  'Jornada 4 – Sábado 13 de junio · 10:00 – 12:00',
  'Jornada 5 – Viernes 19 de junio · 14:00 – 16:00',
  'Jornada 6 – Viernes 19 de junio · 16:40 – 18:40',
  'Jornada 7 – Sábado 20 de junio · 08:55 – 10:30',
];
```

8.5 Lista de panelistas (hardcodeada)

```
const panelistas = [
  // Asisten a TODAS las jornadas
  { nombre: 'Francisco Forero', email: 'fforeromendoza@gmail.com', jornadas: J0
  { nombre: 'Diego Tovar', email: 'diego.tovar@internalconsulting.com', jornadas: J
  { nombre: 'Juan Camilo Camargo', email: 'juan@diboca.co', jornadas: J
  { nombre: 'Natalia Jiménez', email: 'natismonroy@gmail.com', jornadas: J
  { nombre: 'Mariana García', email: 'mariana.garcia@alascinco.org', jornadas: J
  // Asisten a jornadas específicas
  { nombre: 'Fabián Motta Hurtado', email: 'fabian.motta@adabtech.com', jornadas: [
  { nombre: 'Mayrena Barraza', email: 'mayrenabarraza@gmail.com', jornadas: [
  { nombre: 'Rodrigo Ospina', email: 'rodrigo@consultoresog.com.co', jornadas:
  { nombre: 'Juan Manuel Martínez', email: 'j.martinez@organizacionmas.com', jornadas: [
  { nombre: 'Diana Catalina Blanco', email: 'catablancof@gmail.com', jornadas: [
  { nombre: 'Juan Nicolás Piñeros', email: 'juan.pineros@grupopentalia.com', jornadas: [
  { nombre: 'Christian Cabarique', email: 'disenadorcabarique@gmail.com', jornadas: [
];
```

Estructura de cada panelista:

- **nombre** : Nombre completo para mostrar en tabla y emails
- **email** : Clave primaria — se usa para indexar en JSONbin y enviar emails
- **jornadas** : Array de strings de **JORNADAS** a las que asiste (referencia al mismo array)
- **todas** : Boolean — **true** si asiste a todas las 7 jornadas

Helpers de conveniencia:

```
const tieneViernes = p => p.jornadas.some(j => j.includes('Viernes'));
const tieneSabado  = p => p.jornadas.some(j => j.includes('Sábado'));
```


8.6 Sincronización con JSONbin

```
let logistica = {}; // Estado local, se popula desde JSONbin
let modalIdx = -1; // Índice del panelista cuyo modal de logística está abierto

// Leer
async function cargarDatos() {
  const r = await fetch(`${BIN_URL}/latest`, { headers: {'X-Master-Key': BIN_KEY} });
  const j = await r.json();
  logistica = j.record?.logistica || {};
  renderStats(); renderTable();
}

// Escribir
async function guardarEnCloud() {
  await fetch(BIN_URL, {
    method: 'PUT',
    headers: {'Content-Type': 'application/json', 'X-Master-Key': BIN_KEY},
    body: JSON.stringify(logistica)
  });
}

// Acceso al registro de un panelista
function getLogi(email) { return logistica[email] || {}; }
function setLogi(email, data) { logistica[email] = {...getLogi(email), ...data}; }
```

El botón **Actualizar** llama a `cargarDatos()` manualmente. No hay sincronización automática/polling.

8.7 Estadísticas (stats bar)

Cuatro tarjetas calculadas al vuelo desde el estado local:

```
function renderStats() {
  const total = panelistas.length; // Siempre 12
  const enviados = panelistas.filter(p => getLogi(p.email).enviado).length;
  const conTransp = panelistas.filter(p => getLogi(p.email).transporte === 'si').length;
  // Muestra: Total | Confirmaciones enviadas | Necesitan transporte | Pendientes (total-
}
```

8.8 Tabla de panelistas

Columnas: `#` | `Nombre` | `Email` | `Jornadas` | `Transporte` | `Comidas` | `Acciones`

Columna Jornadas:

- Si `todas: true` → badge verde `"TODAS J1-J7"`
- Si `todas: false` → badges rojos `"Jornada N"` por cada jornada asignada

Columna Transporte:

- Sin datos: badge gris `"Sin definir"`
- `transporte: "si"` : badge verde `"✓ Sí"` + línea con fechas + dirección + hora
- `transporte: "no"` : badge rojo `"No"`

Columna Comidas:

- Para panelistas con Viernes: badge por fechas de almuerzo o "Almuerzo?" si sin definir
- Para panelistas con Sábado: badge por fechas de desayuno o "Desayuno?" si sin definir
- Formato de fechas: "12 jun, 19 jun" (abreviado)

Columna Acciones (cuando NO ha recibido confirmación):

[📄 Logística] [✉ Enviar confirmación] [✉ Enviar + logística]

Columna Acciones (cuando YA recibió confirmación):

[📄 Logística] [✓ Confirmación enviada]

8.9 Botones de acción globales

[📄 Ver resumen por jornada] [⬆ Enviar a todos los pendientes]

El botón de "Enviar a todos" está deliberadamente pequeño, con borde fino y opacidad reducida para evitar clics accidentales.

8.10 Modal de logística (admin)

Se abre con `abrirModalLog(idx)` donde `idx` es el índice del panelista en el array.

Secciones:

1. **Transporte:** Select (Sin definir / Sí / No) → al seleccionar "Sí" aparece:
 - Checkboxes de fechas (habilitados solo para las fechas que ese panelista tiene)
 - Campo texto de dirección
 - Campo time de hora
2. **Almuerzo** (solo si el panelista tiene Viernes):
 - Checkbox "Viernes 12 de junio" (si tiene esa fecha)
 - Checkbox "Viernes 19 de junio" (si tiene esa fecha)
 - Los checkboxes no aplicables aparecen con clase `.disabled` y `disabled`
3. **Desayuno** (solo si el panelista tiene Sábado):
 - Checkbox "Sábado 13 de junio"
 - Checkbox "Sábado 20 de junio"

Lógica de checkboxes de almuerzo en apertura:

```
// Determinar qué fechas Viernes tiene este panelista
const almJornadas = p.todas
  ? ['12', '19']
  : p.jornadas.filter(j => j.includes('Viernes')).map(j => j.includes('12') ? '12' : '19')

// Pre-marcar: si hay fechas guardadas úsalas; si no, marcar todas las disponibles
cb.checked = disponible && (fechasAlm.length ? fechasAlm.includes(valor) : true);
```

Guardar:

```
async function guardarLogistica() {
  const transp = document.getElementById('m-transporte').value || null;
  // Recoger fechas transporte: querySelectorAll('input[name="m-fecha"]:checked:not(:disa
  // Recoger fechas almuerzo: querySelectorAll('input[name="m-almuerzo-fecha"]:checked:no
  // Recoger fechas desayuno: querySelectorAll('input[name="m-desayuno-fecha"]:checked:no
  setLogi(p.email, { transporte, fechas_transporte, direccion, hora, fechas_almuerzo, fec
  await guardarEnCloud();
  renderStats(); renderTable();
}
```

8.11 Envío de emails

Enviar confirmación básica (`enviarUno`):

```
async function enviarUno(idx) {
  const jornadaText = p.todas ? TODAS_LABEL : p.jornadas.join('\n');
  await emailjs.send(SVC, TPL, { nombre:p.nombre, email:p.email, jornada:jornadaText });
  setLogi(p.email, { enviado:true });
  await guardarEnCloud();
}
```

Enviar confirmación con logística (`enviarUnoConLogistica`):

```
async function enviarUnoConLogistica(idx) {
  const jornadaText = (p.todas ? TODAS_LABEL : p.jornadas.join('\n'))
    + armarTextoLogistica(log, p); // Appended al campo jornada del template
  await emailjs.send(SVC, TPL, { nombre:p.nombre, email:p.email, jornada:jornadaText });
}
```

`armarTextoLogistica(log, p)` genera un bloque de texto:

```
— Logística confirmada —
🚚 Transporte: Sí
Fechas: Viernes 12 de junio, Sábado 13 de junio
📍 Cra 15 #85-32
🕒 07:30
🍽️ Almuerzo: Sí
🥛 Desayuno: No
```

Enviar a todos los pendientes (`enviarATodos`):

Itera sobre todos los panelistas con `enviado !== true`, envía con un delay de 500ms entre cada uno para no saturar EmailJS, y muestra toast con el conteo al finalizar.

8.12 Resumen por jornada

`mostrarResumen()` genera texto plano organizado así:

Jornada 1 – Viernes 12 de junio · 13:50 – 15:50

- Nombre Panelista
 Transporte – Dirección · HH:MM |  Almuerza
- Otro Panelista
 No almuerza

[...todas las jornadas...]

RESUMEN DE TRANSPORTE

-  17 Viernes 12 de junio
- Nombre Panelista
 Dirección  HH:MM

Total panelistas: 12 | Con transporte: 4
Generado: dd/mm/aaaa, hh:mm:ss

Lógica clave del resumen:

Para cada jornada se extrae la fecha con:

```
const fechaDeJornada = j => j.replace(/^.*- /, '').replace(/ .*$/, '').trim();  
// "Jornada 1 – Viernes 12 de junio · 13:50" → "Viernes 12 de junio"
```

El transporte solo aparece si `fechas_transporte` incluye la fecha de esa jornada (o si `fechas_transporte` es null/vacío, se asume que aplica para todas).

El almuerzo solo aparece si es jornada de Viernes y `fechas_almuerzo` está definido.

El desayuno solo aparece si es jornada de Sábado y `fechas_desayuno` está definido.

8.13 Descarga Excel (CSV)

`descargarExcel()` genera un archivo `.csv` con separador punto y coma (`;`) y BOM UTF-8 (`﻿`) para compatibilidad con Excel en español.

Columnas del CSV:

1. Jornada (texto completo)
2. Fecha (solo la fecha, sin hora)
3. Hora jornada
4. Panelista
5. Transporte esta fecha (`"Sí"` / `"No"` / `"No (otra fecha)"`)
6. Dirección de recogida
7. Hora de recogida
8. Almuerza (`"Sí"` / `"No"` / `""` si no aplica)
9. Desayuna (`"Sí"` / `"No"` / `""` si no aplica)

Una fila vacía separa cada jornada. El archivo se descarga como `NAVES_Panelistas_YYYY-MM-DD.csv`.

8.14 Toast de notificaciones

```
function mostrarToast(msg, error=false) {  
  const t = document.getElementById('toast');  
  t.textContent = msg;  
  t.className = 'toast show' + (error ? ' error' : '');  
  setTimeout(() => { t.className = 'toast'; }, 3500);  
}
```

Aparece en la esquina inferior derecha, fondo verde (éxito) o rojo (error), desaparece a los 3.5 segundos.

9. programador.html — Generador de Programación

Página agregada después de la versión inicial. Es la primera pieza que materializa, de forma práctica, el "módulo de calendario" descrito conceptualmente en §16.10. Documenta un generador real y funcionando, no una propuesta.

9.1 Propósito y audiencia

`programador.html` es una herramienta de **uso exclusivo del coordinador / profesor que arma la programación** del evento. Reemplaza el proceso manual de construir a mano el Excel de programación (el archivo `Programacion NAVES FS 24-26.xlsx` que se mantenía a mano, con una hoja por día).

A partir de unos parámetros, el generador **construye el cronograma completo** (presentaciones + foto inicial + introducción + breaks + almuerzos + cierre) y produce dos entregables:

1. **Cronograma imprimible (vista web)** — para participantes y asistentes de programa; se pega en carteleras.
2. **Excel de calificación de panelistas** — una hoja por día, con toda la programación del día y casillas en blanco para que cada panelista califique a mano.

URL: `https://navesfs.netlify.app/programador.html`

Acceso: protegido con contraseña (`Mba2026` , la misma del admin — ver §9.7).

Enlace de entrada: botón  "Generador de programación" en la barra de acciones de `admin.html`.

9.2 Diferencia clave frente a `index.html` y `naves_generator.py`

Pieza	Qué hace con la programación
<code>index.html</code>	Muestra el cronograma ya decidido (orden y horas escritos a mano en el HTML)
<code>naves_generator.py</code>	Genera <code>index.html</code> agrupando por sector — no sabe de horarios
<code>programador.html</code>	Calcula el cronograma (horas, breaks, almuerzos) a partir de parámetros, con recálculo en cascada

Es decir: el orden de quién presenta lo decide el usuario; el generador solo **acomoda ese orden en la grilla de tiempos** y resuelve dónde caen breaks, almuerzos y fotos.

9.3 Fuente de los proyectos y las fechas (punto de integración)

Hoy el generador trae **precargados** dos conjuntos de datos que, a futuro, deben **recibirse del sistema naves-inalde.com** (no inventarse aquí):

Dato	Hardcodeado hoy	Origen futuro
Proyectos (<code>proyecto</code> , <code>autores</code> , <code>sector</code>)	<code>PROYECTOS_BASE</code> (copiado de <code>index.html</code>)	API de la cohorte (§16.6.6)
Fechas de presentación (días + hora de inicio)	<code>DEFAULTS</code>	API de la cohorte (§16.6.6)

Lo que **sí** decide el profesor en el programador: el **orden** de quién presenta cuándo, los **breaks/almuerzos** y las **horas**. Eso no viene de afuera.

La plataforma destino ya existe y su API se verificó (ver §16.6.6 para el formato real: `GET /api/admin/cohortes` , backend Supabase, auth JWT). El **importador JSON** (§9.6b) es el mecanismo concreto para recibir esos datos: cuando se confirme el formato del detalle de cohorte (proyectos + fechas de presentación), se mapea a la entrada `{ proyectos:[...], fechas:[{fecha, inicio}] }` y se reemplazan `PROYECTOS_BASE` y `DEFAULTS` . Este es el único punto de integración pendiente.

9.4 Modelo de estado

```
let estado = {
  tipo: 'FS',          // 'FS' (fin de semana) | 'INT' (intensivo)
  dias: [
    {
      fechaISO: '2026-06-12',      // FECHA REAL – define el orden cronológico
      fecha:    'Viernes 12 de junio', // título DERIVADO de fechaISO (fechaTitulo())
      inicio:   '13:50',           // hora de la 1ª PRESENTACIÓN (no de la foto)
      foto:     true,              // ¿hay foto inicial en la puerta?
      intro:    20,                // minutos de introducción
      n:        10,                // nº de proyectos asignados a este día
      interr:   [ {tipo:'break', min:30, auto:true}, ... ] // una por boundary interno
    },
    ...
  ],
  orden: [...],           // copia reordenable de los proyectos (§9.5b)
  solicitudes: [...]     // solicitudes especiales, máx 4 (§9.5b)
};
```

Orden cronológico: cada día tiene una **fecha real** (`fechaISO` , formato `YYYY-MM-DD`) que es la fuente de verdad del orden. El título visible (`fecha`) se **deriva** de ella con `fechaTitulo()` (ej. "Viernes 12 de junio"). En cada recálculo, `ordenarDias()` ordena los días por `fechaISO` y luego por hora de inicio — así, **al agregar un día se ubica solo en su posición cronológica** (`agregarDia()`) propone la fecha del día siguiente al último). La UI usa un selector de fecha (`type=date`), no texto libre.

Hay defaults por tipo de programa en la constante `DEFAULTS` (FS = 4 días viernes/sábado de junio 2026; INT = lun/mar/mié 16–18 nov 2026). Cambiar el tipo recarga los defaults de ese tipo.

9.5 Constantes de tiempo

Tomadas de la hoja `TIEMPOS` de los Excel históricos (idénticas entre Intensivo y FS):

Constante	Default	Editable en UI
Exposición	20 min	sí (<code>t-expo</code>)
Transición entre slots	5 min	sí (<code>t-trans</code>)
Foto inicial	10 min	sí (<code>t-foto</code>)
Cierre / foto final	20 min	sí (<code>t-cierre</code>)
Introducción	por día	sí (campo por día)
Break / Almuerzo	por interrupción	sí (campo por interrupción)

Otras constantes en código: `BLOQUE = 5` (presentaciones por bloque) y `VENTANA_ALM = [690, 870]` (ventana de almuerzo 11:30–14:30 en minutos desde medianoche).

9.5b Reorden de presentaciones, solicitudes especiales y auditoría

El orden de quién presenta lo decide el profesor. La herramienta lo soporta así:

Orden reordenable (`estado.orden`). Es una copia de `PROYECTOS_BASE` que el usuario reordena. El número de slot = posición + 1; el día se asigna por el conteo `n` de cada día. Toda la app (preview, Excel, reorden, auditoría, solicitudes) lee de `computarTodo()`, que es la única fuente de verdad: devuelve `{dias, slots}` ya con horas calculadas.

Tres formas de reordenar (panel "Reordenar presentaciones"):

- **Arrastre con pointer events** (mouse y táctil): se inicia desde el handle `onpointerdown="dragStart"`. Durante el arrastre se muestra (a) un **ghost flotante** con el nombre del proyecto y el **slot de destino en vivo** ("→ slot N"), y (b) una **línea roja** que marca dónde caerá. Hay **auto-scroll** al acercar el puntero a los bordes de la lista. Al soltar (`dragEnd`) se llama a `moverProyecto(from,target)`. Se reemplazó el drag-and-drop nativo de HTML5 porque era errático (los eventos saltaban entre los hijos de cada fila y no daba realimentación visual ni soporte táctil).
- **Botones ▲ ▼** por fila.
- **"Mover a slot #"**: input numérico (`moverASlot`).

La lista muestra **separadores de día** (`.ro-day`, sticky) para orientarse sobre en qué día cae cada proyecto mientras se reordena.

Cualquier movimiento llama a `recomputar()`, que recalcula **toda** la programación y vuelve a correr la auditoría.

Agrupar por sector (`agruparPorSector()`): reordena `estado.orden` según `SECTOR_AFINIDAD` (orden de cercanía entre sectores), como punto de partida; pide confirmación porque descarta el orden manual. La afinidad por defecto es: FinTech/Financiero → IA/Tecnología → HealthTech/Salud → Salud/Deporte → Alimentos/F&B → AgriTech/Sostenibilidad → Movilidad/Energía → RRHH/Bienestar → Otros.

Solicitudes especiales (`estado.solicitudes` , máx. 4). Cada una = `{proyecto, tipo, detalle, nota}` . Tipos (`TIPOS_SOLICITUD`):

- `temprano` — presenta en el primer bloque de su día (antes del 1er break/almuerzo).
- `tarde` — presenta en el último bloque de su día (después del último break/almuerzo).
- `dia` — cae en un día específico (`detalle` = fecha).
- `hora` — la hora real está dentro de ± 30 min de la pedida (`detalle` = hora).

`evaluarSolicitud()` compara la solicitud con la programación actual y devuelve `✓` (verde) o `△` (naranja) con el estado real. **El sistema solo señala; nunca reordena solo** — el control de los slots es del profesor.

Auditoría (`renderAuditoria()`): en cada recálculo verifica integridad y muestra `✓` o la lista de problemas. Chequea: proyectos duplicados, proyectos faltantes, suma de `n` por día vs. total, días con 0 proyectos, solapamientos de hora dentro de cada día, y numeración de slots consecutiva (1..N).

9.6 El motor: `construirDia()`

Núcleo del sistema. Dado un día y su lista de proyectos, devuelve un arreglo de filas (`{tipo, ini, fin, ...}`):

1. **Foto + introducción se calculan hacia atrás** desde la hora de la 1ª presentación:
`t_inicio_foto = inicio - transición - intro - (foto ? fotoMin : 0)`
Así, cuando el usuario escribe "13:50" obtiene exactamente lo que ve en el sitio publicado (foto 13:15, intro 13:25, slot 1 a las 13:50).
2. Cada presentación ocupa `exposición` minutos, seguida de `transición` .
3. **Cada 5 presentaciones (bloque)** se inserta una interrupción con la foto de esos grupos:
 - Si **no** es el último bloque del día → break o almuerzo (según `interr[]`).
 - Si **es** el último bloque → "Cierre de jornada" (o "Evaluación y Cierre" el último día).
4. Cualquier cambio en un campo dispara `onCfg() → recomputar()` , que recalcula **todos los días en cascada** y re-renderiza.

Asignación automática break vs. almuerzo (`sincronizarInterrupciones()`): cada interrupción nace con `auto:true` . En cada recálculo, si su hora de inicio cae dentro de `VENTANA_ALM` se marca como **almuerzo (60 min)**, si no como **break (30 min)**. Cuando el usuario cambia manualmente el tipo o los minutos, se fija `auto:false` y su decisión se respeta de ahí en adelante.

Nota de fidelidad: el motor usa una transición uniforme de 5 min. Las hojas históricas tenían pequeñas inconsistencias manuales (ej. 4 min de preparación en el Intensivo, breaks de 25 min). El generador prioriza consistencia; el resultado del modo FS reproduce **exactamente** el cronograma publicado en `index.html` .

9.6b Persistencia (fuente única de la verdad)

El programador es la **fuentes única del orden y el horario**. La programación se guarda en tres capas:

1. **Auto-guardado local** (`localStorage` , clave `naves_programacion_v1`): inmediato en cada cambio; respaldo offline.

2. **Nube compartida** (JSONbin propio, `PROG_BIN_ID = 6a2f33e2da38895dfec095a4`, separado del bin de logística): guardado con *debounce* de 1.5 s tras el último cambio (`programarGuardadoNube` → `guardarNube`). Permite que **varios coordinadores** editen la misma programación. Al abrir, `entrar()` carga primero de la nube (`cargarNube`); si está vacía o falla, usa el respaldo local. Botón "📁 Cargar nube" para traer la última versión (por si otro coordinador la editó). El indicador `#save-ind` muestra el estado (🔒 Guardado / Guardando / Sin conexión).
3. **Export/Import JSON** (ver abajo): respaldo portátil y puente hacia `naves_generator.py` .

Concurrencia: el guardado en la nube usa GET-then-PUT (no atómico), igual que la logística (§4). Para un editor es seguro; con varios simultáneos aplica *last-write-wins*. El botón "📁 Cargar nube" mitiga al permitir refrescar antes de editar.

`snapshot()` / `restore()` serializan/restauran todo el estado: `evento` , `tipo` , `tiempos` , `dias` , `orden` , `solicitudes` .

Exportar/Importar programación (JSON). `exportarJSON()` descarga un archivo que incluye el `snapshot` más el `cronograma ya calculado` (cada proyecto con día, slot, hora inicio/fin). Ese `cronograma` es lo que debe consumir `naves_generator.py` para regenerar `index.html` en orden de presentación — el mecanismo concreto para que el orden viva en un solo lugar. `importarJSON()` restaura desde un archivo exportado.

9.7 Autenticación

Igual que `admin.html` : contraseña en cliente (`const PWD = 'Mba2026'`). Mismas limitaciones de seguridad de §13.2 — adecuado para prototipo interno, debe reemplazarse por autenticación real en producción.

9.8 Salidas

Cronograma imprimible — `window.print()` con CSS `@media print` que oculta configuración, paneles (`.no-print`) y botones, y deja solo la vista previa. **Un día por página:** cada `.dia-prog` lleva `break-after:page` (el último, `break-after:auto` , para no dejar una hoja en blanco al final). `@page{margin:1.2cm}` . Para guardar PDF: imprimir → destino "Guardar como PDF".

Excel de calificación (`exportarExcel()` , vía **ExcelJS**) — un workbook con **una hoja por día**.

Se usa **ExcelJS** (no SheetJS): la versión gratuita de SheetJS **no soporta estilos** (colores, fuentes, bordes). ExcelJS sí, en el navegador. Se carga desde CDN:

`https://cdnjs.cloudflare.com/ajax/libs/exceljs/4.4.0/exceljs.min.js` . La función es `async` (usa `workbook.xlsx.writeBuffer()` + descarga vía Blob).

Cada hoja:

- Título del evento + fecha + fila **Panelista:** con línea inferior para escribir el nombre a mano.
- Tabla con **toda la programación del día** (proyectos + actividades), porque es la guía del panelista durante las sesiones.
- Columnas: `Slot` | `Inicio` | `Fin` | `Proyecto/Actividad` | `Autores` | `Calif. presentación (1-5)` | `Calif. proyecto (1-5)` | `¿Invertiría? (Sí/No)` .

- **Formato:** fuente **tamaño 24**; encabezado negro con texto blanco; **filas de actividad en amarillo #FFE066** (foto, intro, break, almuerzo, cierre); proyectos en blanco; **borde de cuadrícula** en toda la tabla; filas de proyecto altas (100 pt) para escritura a mano.
- **Columnas de calificación continuas:** la descripción de cada actividad se fusiona **solo en D-E** (Proyecto/Autores); las columnas F, G, H bajan como cajas continuas (sin interrumpirse en las filas de actividad). Las casillas de calificación van **vacías** (el panelista escribe el 1–5 a mano; el encabezado indica la escala).

9.9 Tema visual de las franjas de actividad

Las **filas de actividad** (no presentaciones) usan **amarillo #FFE066** de forma consistente en las tres páginas, igual que el Excel:

- `programador.html` : `tr.ev td{background:#ffe066}` (vista previa).
- `index.html` : las 13 filas `evento-row` (estilo inline `background:#ffe066`).
- `panelistas.html` : `.evento-jor{background:#ffe066}`.

Las presentaciones quedan en blanco, para distinguirlas a simple vista. (El cambio en `index.html` y `panelistas.html` fue solo de color de fondo — verificado por diff antes de publicar a producción.)

9.10 Recomendaciones para la versión profesional

- **Conectar PROYECTOS_BASE y las fechas a la API de la cohorte** (§16.6.6) en lugar de los datos precargados.
- **Generar el Excel en el servidor** para volúmenes grandes y formato aún más rico.
- **Persistir en la BD de la plataforma** (tabla `slot_presentacion`, §16.4) en lugar de JSONbin, y alimentar tanto a `index.html` como al portal de participantes (§16.16).
- **Auto-scroll y arrastre entre días** en el reorden (§9.5b) ya implementados; falta drag entre la lista y el calendario visual (§16.10).

10. Generador de Cohorte (prototipo — en pausa hasta migrar a Supabase)

Esta sección documenta un prototipo que se construyó y luego **se dejó en pausa** por una limitación de plan de Netlify (explicada abajo). El código quedó en el repo como punto de partida para cuando el sistema migre a la plataforma `naves-inalde.com` (Supabase). **No está en producción** — solo se desplegó en borradores.

10.1 Objetivo

Generar, para una cohorte nueva, dos cosas a partir del **calendario académico (PDF)**:

1. El **cronograma de 13 hitos** del trabajo de grado (Kick Off, anteproyecto, ventanas R1/R2, reunión "60 días", entrega final, presentaciones, etc.).

2. El **sitio web de información de la cohorte** (timeline + FAQ + formulario), que hoy se genera con la skill `SKILL-website-naves` y se publica aparte (ej. `prueba-int2026-naves.netlify.app`).

La lógica de referencia ya existe (hecha por el usuario, fuera de este repo, en `~/Documents/Claude/Projects/NAVES - INALDE/`):

- `calcular-hitos-naves.py` — calcula los 13 hitos a partir de Kick Off + Ancla + jornadas de clase + festivos (usa la librería `holidays` de Colombia).
- `SKILL-programacion-naves.md` — reglas de los 13 hitos (días hábiles, Semana Santa, ajustes), con 3 checkpoints de confirmación.
- `SKILL-website-naves.md` + `template-website-naves.html` — genera el `index.html` de la cohorte (24 placeholders, SHA-256, Web3Forms).

10.2 Qué se construyó (en borrador)

Archivo	Función
<code>generador-cohorte.html</code>	Página protegida (contraseña <code>Mba2026</code>). Paso 1: subir el PDF del calendario → leerlo con Claude.
<code>netlify/functions/extraer-calendario-background.js</code>	Función que manda el PDF a la API de Claude (visión) y guarda el resultado en JSONbin.
<code>netlify/functions/resultado-calendario.js</code>	Función de consulta del resultado por <code>jobId</code> .
<code>netlify.toml</code>	Habilita el directorio de funciones.

- **JSONbin de resultados:** `6a302c5ada38895dfec095a4` ... (bin `NAVES Extraccion Calendario` , dict `jobs` keyed por `jobId`).
- **Credencial:** `ANTHROPIC_API_KEY` está configurada como **variable de entorno secreta en Netlify** (no en el código). Es una API key de Anthropic (`console.anthropic.com`), saldo prepago (~\$5, alcanza años para 3 usos/año).
- **Extracción (verificada):** Claude lee el calendario color-coded y devuelve JSON: `tipo_detectado` (Intensivo/Fin de Semana), `anio` , `jornadas_clase` (amarillo), `festivos` (rojo), `semana_santa_lunes` , `presentaciones` (celdas "Presentación NAVES" → el **Ancla**), `grado` , `eventos` , `notas` .

10.3 La limitación que lo dejó en pausa (importante)

Leer un calendario visual de varios años es una **tarea de visión/IA** que requiere un modelo fuerte. Se midió en vivo:

Modelo	Tiempo en función Netlify	Resultado
Haiku 4.5	~16 s ✓ (cabe en el límite)	Lee, pero se equivoca en el AÑO de algunas celdas (puso presentaciones en 2025/noviembre en vez de junio 2026)
Sonnet 4.6	~31 s ✗	504 Inactivity Timeout (límite síncrono ≈ 26 s)
Opus 4.8	aún más lento ✗	No cabe en síncrono
Cualquiera en función de fondo (- background , sin límite de tiempo)	—	HTTP 500 : las funciones de fondo requieren el plan Pro de Netlify (\$19/mes) ; el sitio está en un plan de \$9 que no las incluye

Conclusión: en el plan actual, dentro del sitio solo cabe Haiku (impreciso). Los modelos buenos (Sonnet/Opus) superan el límite de tiempo síncrono y necesitan funciones de fondo, que requieren plan Pro. Por eso la lectura **funciona perfecto en el chat de Claude** (corre Opus sin límite) y no en una función de Netlify de bajo plan.

10.4 Decisión y camino recomendado

Se **difirió la lectura automática en el sitio** hasta que NAVES migre a la plataforma **naves-inalde.com (Supabase)**. Las **Edge Functions de Supabase** permiten ejecuciones largas en plan gratuito → ahí sí se puede correr **Opus** para leer el PDF con la calidad del chat. Ese es el hogar natural de esta función (ver §16.6).

Mientras tanto, el flujo confiable es: el coordinador da el PDF a Claude **en el chat** (Opus lee con calidad, como la skill ya validada), confirma las fechas clave en los checkpoints, y la parte determinista (calcular los 13 hitos + generar el sitio) se hace con la herramienta. El **Kick Off** y el **Ancla** siempre los confirma el coordinador (igual que en **calcular-hitos-naves.py**, donde son inputs manuales) — no se confía ciegamente en la extracción.

10.5 Para retomar (cuando se migre a Supabase)

1. Reescribir **extraer-calendario-background.js** como **Supabase Edge Function** (Deno) con el mismo prompt y esquema; modelo **claude-opus-4-8** (sin límite de tiempo de fondo).
2. La API key como **secreto de Supabase** (**supabase secrets set ANTHROPIC_API_KEY=...**).
3. Conectar el **generador-cohorta.html** (o su equivalente en la plataforma) a esa Edge Function.
4. Construir los pasos 2 (calculador de 13 hitos en JS, portando **calcular-hitos-naves.py** con festivos de Colombia) y 3 (generador del sitio con **template-website-naves.html**).
5. Mantener los **checkpoints de confirmación** del coordinador para Kick Off, Ancla y Semana Santa.

11. Diseño visual — Sistema de tokens CSS

Ambas páginas comparten el mismo sistema de tokens CSS:

```
:root {
  --red:    #e30613; /* INALDE rojo corporativo */
  --black:  #0a0a0a; /* Fondo header/hero */
  --text:   #1a1a1a; /* Texto principal */
  --gray:   #6b6b6b; /* Texto secundario */
  --light:  #f5f5f5; /* Fondo de página */
  --border: #e0e0e0; /* Bordes */
  --white:  #ffffff;
  --green:  #1a6b3c; /* Solo en admin.html */
}
```

Tipografías: Montserrat (títulos, badges, botones) + Roboto (cuerpo, formularios)

Fuente: Google Fonts

12. Despliegue

Requisitos

- Cuenta en Netlify (netlify.com)
- Netlify CLI instalado: `npm install -g netlify-cli`
- Login: `netlify login` (OAuth con jmvicaria@gmail.com)

Comando de despliegue

```
cd /Users/juanmanuel/Desktop/NAVES_2026_Web
netlify deploy --prod --dir .      # PRODUCCIÓN (publica a navesfs.netlify.app)
netlify deploy --dir .             # BORRADOR (URL de preview única; NO toca producción)
```

No hay build step. El directorio completo se despliega tal cual. Los deploys de Netlify son **atómicos** (cambio instantáneo del sitio completo); republicar un archivo idéntico no afecta a quien lo esté usando.

Despliegue seguro mientras hay usuarios en vivo (flujo borrador)

Mientras `index.html` y `panelistas.html` están **en uso en producción**, se desarrolla cualquier módulo nuevo (ej. `programador.html`) con **deploys borrador**, dejando producción congelada:

1. Iterar con `netlify deploy --dir .` → genera una **Draft URL** única para revisar/probar. Producción no cambia.
2. Solo al aprobar, hacer **un** `netlify deploy --prod --dir .`
3. Antes de ese `--prod`, verificar que `index.html` y `panelistas.html` no cambiaron respecto a lo que está en vivo (un diff), para garantizar que no se altera la experiencia de los usuarios conectados.

Riesgo real = **editar** esos archivos, no desplegar. Por eso el flujo borrador + verificación de diff antes de promover. Ver memoria de proyecto [naves-deploy-borrador](#) .

Estructura de archivos del proyecto

```
NAVES_2026_Web/
├── index.html           Base de datos de proyectos – interno INALDE (§6)
├── panelistas.html      Página pública de panelistas (§7)
├── admin.html           Panel de administración (§8)
├── programador.html     Generador de programación (§9)
├── logos/               Logos de los 34 emprendimientos
│   ├── Bevo.jpg
│   ├── SATORI.jpg
│   └── ...
├── pdfs/                One Pagers en PDF
│   ├── Bevo_resumen.pdf
│   └── ...
├── og-panelistas.jpg    Imagen Open Graph (WhatsApp preview)
├── netlify.toml         (si existe) Configuración de Netlify
└── DOCUMENTACION_TECNICA.md Este documento
```

El generador `naves_generator.py` (§5) vive fuera de esta carpeta, en `/Users/juanmanuel/Dropbox/INALDE/NAVES/NAVES_Agentes/` ; produce `index.html` y copia logos/PDFs aquí.

13. Limitaciones del prototipo y recomendaciones para la versión profesional

13.1 Credenciales expuestas en el cliente

Problema: Las API keys de JSONbin y EmailJS están visibles en el código fuente del navegador.

Solución: Mover toda la comunicación con APIs a funciones serverless (Netlify Functions, Vercel Edge Functions, o un back-end propio). El cliente solo habla con el propio back-end, que guarda las keys en variables de entorno.

13.2 Sin autenticación real

Problema: La contraseña de admin es visible en el código fuente (`const PWD = 'Mba2026'`).

Solución: Implementar autenticación real. Opciones:

- **Netlify Identity** (gratuito, integrado con Netlify)
- **Supabase Auth** (gratuito, robusto)
- **Auth0** (gratuito hasta 7000 usuarios activos/mes)

13.3 Lista de panelistas hardcodeda

Problema: Para cambiar los 12 panelistas preconfigurados hay que editar el HTML y redespargar.

Mitigación implementada (10 jun 2026): El admin ahora detecta automáticamente registros nuevos. Cuando alguien confirma en `panelistas.html`, se guarda en JSONbin su `nombre`, `jornada` y `jornadas_confirmadas` (array que acumula múltiples confirmaciones individuales de la misma persona). El admin combina los 12 hardcodedos con cualquier email extra encontrado en JSONbin (`actualizarListado()`) y los muestra con la insignia "NEW Nuevo registro". Los nuevos participan de las estadísticas, el resumen por jornada, el Excel y los botones de envío de email.

Limitación restante: Si la persona confirma pero el navegador falla al escribir en JSONbin (red, JS bloqueado), su registro queda solo en Netlify Forms y no aparece en el admin. Los registros de Netlify Forms se pueden consultar con `netlify api listFormSubmissions --data '{"form_id":"..."}'`.

Solución definitiva: Mover el catálogo de panelistas a la misma base de datos (JSONbin, Supabase, Airtable, etc.). El admin podría agregar/quitar panelistas desde la interfaz.

13.4 Sin concurrencia ni transacciones

Problema: El patrón GET-then-PUT no es atómico. Datos pueden sobrescribirse si dos usuarios guardan simultáneamente.

Solución: Usar una base de datos que soporte operaciones atómicas y en tiempo real:

- **Supabase** (PostgreSQL + Realtime)
- **Firebase Firestore** (NoSQL + Realtime)

13.5 Sin historial de cambios

Problema: No hay log de quién cambió qué y cuándo.

Solución: Implementar un sistema de auditoría básico (agregar campo `updated_at` y `updated_by` a cada registro).

13.6 Integración con plataformas INALDE

Para integrar con las plataformas existentes de INALDE (donde se gestiona el trabajo de grado):

- **API REST:** Exponer los datos de logística como API para que otras plataformas los consuman
- **Webhooks:** Cuando un panelista confirma → trigger a plataforma INALDE con nombre, email, jornadas
- **SSO:** Si INALDE tiene Microsoft 365 o Google Workspace, usar OAuth para que el admin no necesite contraseña separada
- **Base de datos compartida:** Reemplazar JSONbin por la base de datos que ya usa INALDE (probablemente SQL Server, MySQL, o similar)

13.7 Proyectos y jornadas hardcodedos

Problema: Los 34 proyectos, logos, PDFs y estructura de jornadas están embebidos en el HTML. Cualquier cambio requiere editar HTML y redespargar.

Solución: Cargar los proyectos desde una base de datos o CMS:

- **Airtable** como CMS (tiene API REST gratuita)
- **Notion API** (si INALDE ya usa Notion)
- **CMS headless** (Contentful, Sanity, Strapi)

14. Flujos de usuario completos

Flujo panelista — Confirmación individual

1. Panelista recibe link por email/WhatsApp
2. Abre panelistas.html en el navegador
3. Lee instrucciones en el hero
4. Revisa proyectos de la jornada de interés
5. Descarga One Pagers relevantes (opcional)
6. Clic en "👉 Quiero asistir a esta jornada"
7. Formulario se despliega debajo del botón
8. Ingresa nombre y email → clic en "Confirmar"
9. Sistema envía registro a Netlify Forms (silencioso)
10. Sistema envía email de confirmación via EmailJS (silencioso)
11. Formulario se oculta, aparece modal de logística
12. En modal:
 - a. Selecciona si necesita transporte
 - Si sí: ve la fecha de esa jornada automáticamente
ingresa dirección y hora preferida
 - b. Si es jornada de Viernes: marca si almuerza ese Viernes
 - c. Si es jornada de Sábado: marca si desayuna ese Sábado
13. Clic en "Guardar preferencias"
14. Sistema guarda en JSONbin (merge con datos existentes)
15. Modal se cierra, aparece mensaje "✅ Asistencia confirmada"
16. Botón de confirmación queda deshabilitado y verde

Flujo panelista — Confirmación "Todas las jornadas"

- (Igual al anterior pero con el banner en la parte superior)
- En el modal de logística:
- Sección transporte muestra checkboxes de las 4 fechas (todas marcadas)
 - Panelista puede desmarcar las fechas en que NO necesita transporte
 - Sección almuerzo muestra checkboxes de Viernes 12 y Viernes 19
 - Sección desayuno muestra checkboxes de Sábado 13 y Sábado 20

Flujo administrador — Gestión diaria

1. Abre `admin.html`
2. Ingresa contraseña "Mba2026"
3. Sistema carga datos de JSONbin y renderiza tabla
4. Revisa estadísticas: total, confirmados, transporte, pendientes
5. Para editar logística de un panelista:
 - a. Clic en "Logística"
 - b. Modal se abre con datos actuales del panelista
 - c. Modifica lo necesario (transporte, fechas, dirección, hora, comidas)
 - d. Clic en "Guardar" → se actualiza JSONbin, se re-renderiza tabla
6. Para enviar email de confirmación:
 - a. Clic en "Enviar confirmación" → email básico con jornadas
 - b. Clic en "Enviar + logística" → email con jornadas + bloque de logística
 - c. Botón cambia a "Confirmación enviada" (verde, no clickeable)
7. Para ver resumen:
 - a. Clic en "Ver resumen por jornada"
 - b. Modal muestra texto por jornada con nombre + transporte + comidas
 - c. Clic en "Copiar" → clipboard
 - d. Clic en "Excel" → descarga CSV
8. Clic en "Actualizar" para refrescar datos desde JSONbin

15. Preguntas frecuentes para el próximo desarrollador

¿Por qué está todo en un solo HTML sin framework?

Decisión deliberada de velocidad. No hay build step, no hay dependencias npm, no hay webpack. El despliegue es `netlify deploy --prod --dir .` y funciona. Para una versión de producción con múltiples colaboradores se recomienda React, Vue, o SvelteKit.

¿Por qué JSONbin y no una base de datos real?

JSONbin era la opción más rápida de implementar sin back-end. Un solo documento JSON de ~5KB es suficiente para 12 panelistas. En producción usar Supabase, Firebase, o la base de datos que ya tenga INALDE.

¿Por qué EmailJS y no SMTP directo?

Sin back-end no hay forma de enviar emails sin exponer credenciales SMTP. EmailJS es la solución estándar para envío de emails desde el navegador. En producción mover a un servicio como SendGrid o Amazon SES con back-end.

¿Por qué X-Master-Key y no X-Access-Key ?

JSONbin tiene dos tipos de llaves. La llave que empieza con `$2a$10$...` es una **Master Key**. El header correcto es `X-Master-Key`. Usar `X-Access-Key` con una Master Key resulta en error 401 silencioso (los datos parecen guardarse pero no lo hacen).

¿Cómo agrego un nuevo panelista?

En `admin.html` agregar una línea al array `panelistas` con nombre, email, jornadas (referencias al array `JORNADAS[n]`) y `todas` . Si el panelista también va a poder confirmar desde `panelistas.html` , ese archivo no necesita cambios (el email se guarda en JSONbin con cualquier email que ingrese el panelista en el formulario).

¿Cómo cambio la contraseña de admin?

En `admin.html` , buscar `const PWD = 'Mba2026'` y cambiar el valor. Redesplegar a Netlify.

16. Hoja de ruta hacia una plataforma multi-cohorte integrada con naves-inalde.com

Esta sección es un brief técnico para el desarrollador que construirá la versión profesional del sistema. Describe qué debe cambiar, por qué, y cómo debería quedar arquitectado para servir múltiples cohortes del NAVES dentro de la plataforma <https://naves-inalde.com/>

16.1 Diagnóstico: qué está hardcodeado hoy y por qué es un problema

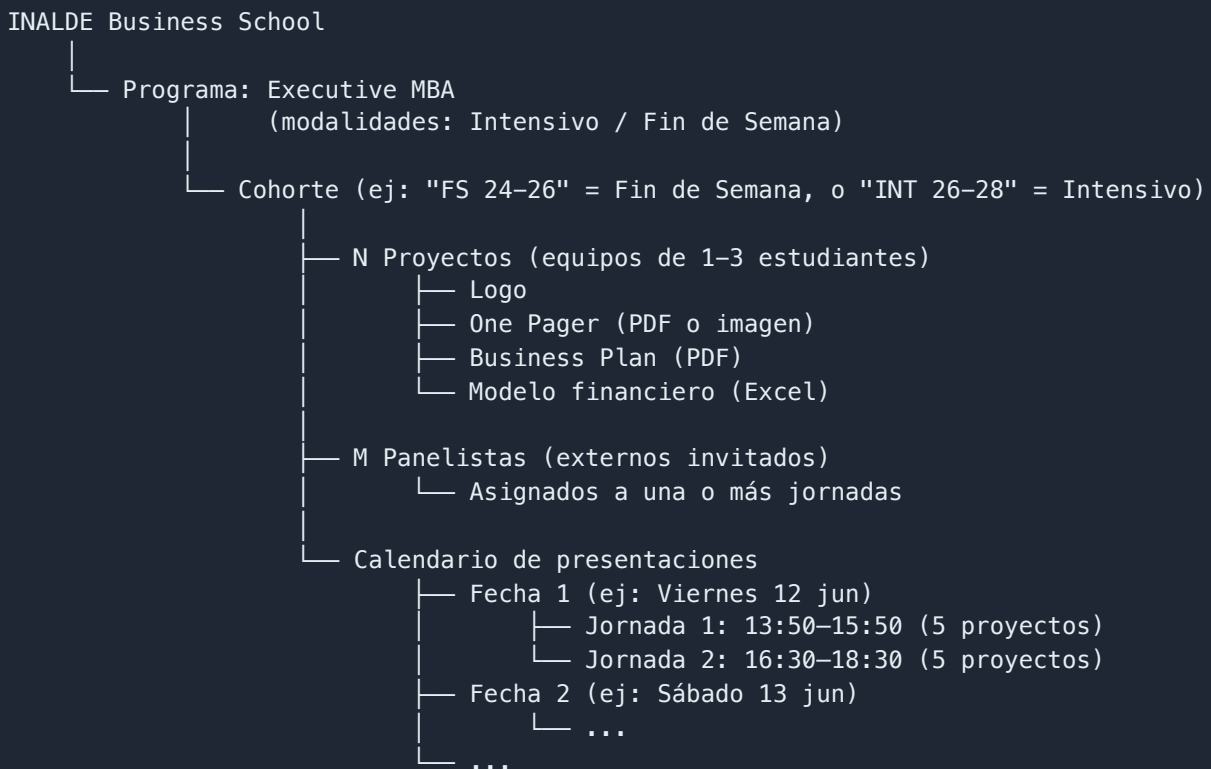
El prototipo actual funciona para una única cohorte (MBA FS 2026) porque toda la información específica de esa edición está embebida directamente en el código fuente. La siguiente tabla detalla cada elemento hardcodeado, dónde vive, y qué implicación tiene al querer reusar el sistema:

Elemento hardcodeado	Ubicación en el código	Problema para multi-cohorte
Fechas de presentación (12, 13, 19, 20 jun 2026)	<code>panelistas.html</code> — <code>FECHAS_MAP</code> ; <code>admin.html</code> — <code>FECHAS_MAP</code> , <code>JORNADAS</code>	Cambiar de año o de fechas requiere editar HTML y lógica JS manualmente
7 jornadas con horarios exactos	<code>admin.html</code> — array <code>JORNADAS</code> (7 strings fijos)	Cohortes futuras pueden tener más o menos jornadas, o distinto número de slots por día
34 proyectos con sus textos, logos y PDFs	<code>index.html</code> — filas <code><tr></code> con datos embebidos como atributos <code>data-*</code>	Imposible actualizar sin editar HTML; no hay separación datos/presentación
Resúmenes y posts LinkedIn de 34 proyectos	<code>naves_generator.py</code> — <code>SUMMARIES_DATA</code> (220 líneas hardcodeadas)	El contenido editorial no puede editarse sin tocar el script Python
12 panelistas con nombres, emails y asignaciones	<code>admin.html</code> — array <code>panelistas</code>	Cada cohorte tiene panelistas distintos; agregar o quitar uno requiere editar y redespargar
Contraseña de admin	<code>admin.html</code> — <code>const PWD = 'Mba2026'</code>	Visible en código fuente; cambia por cohorte; no distingue entre múltiples admins
Credenciales de servicios externos (JSONbin, EmailJS)	<code>panelistas.html</code> y <code>admin.html</code> — constantes al inicio del <code><script></code>	Expuestas en el cliente; si se rota una key hay que editar y redespargar los dos archivos
Sector de cada proyecto	<code>naves_generator.py</code> — dict <code>SECTORES</code>	Requiere que el desarrollador edite el script cada vez que entra un proyecto nuevo
Asignación proyecto→slot→jornada	<code>index.html</code> — <code>data-day</code> en cada <code><tr></code> , editado manualmente	No hay ninguna fuente de verdad; el orden del calendario existe solo en el HTML
Template de email de confirmación	EmailJS dashboard (externo) + texto en <code>armarTextoLogistica()</code>	El texto del email está parcialmente en EmailJS y parcialmente en el JS del admin

Conclusión: El sistema actual es un prototipo de un solo uso. Es totalmente válido para lanzar rápido en 2026, pero no es reutilizable sin una intervención técnica equivalente a rehacerlo. La versión profesional debe separar completamente la **configuración por cohorte**, los **datos de proyectos y panelistas**, y el **código de la aplicación**.

16.2 Modelo conceptual multi-cohorte

Antes de hablar de tecnología, es útil entender el modelo de negocio:



Cada **cohorte** tiene sus propias fechas, sus propios proyectos, y sus propios panelistas. Lo que se mantiene estable entre cohortes es: la identidad visual INALDE/NAVES, el flujo de trabajo (confirmación de panelistas, logística, base de datos pública), y las reglas de negocio (un panelista puede ir a una o más jornadas, puede necesitar transporte, etc.).

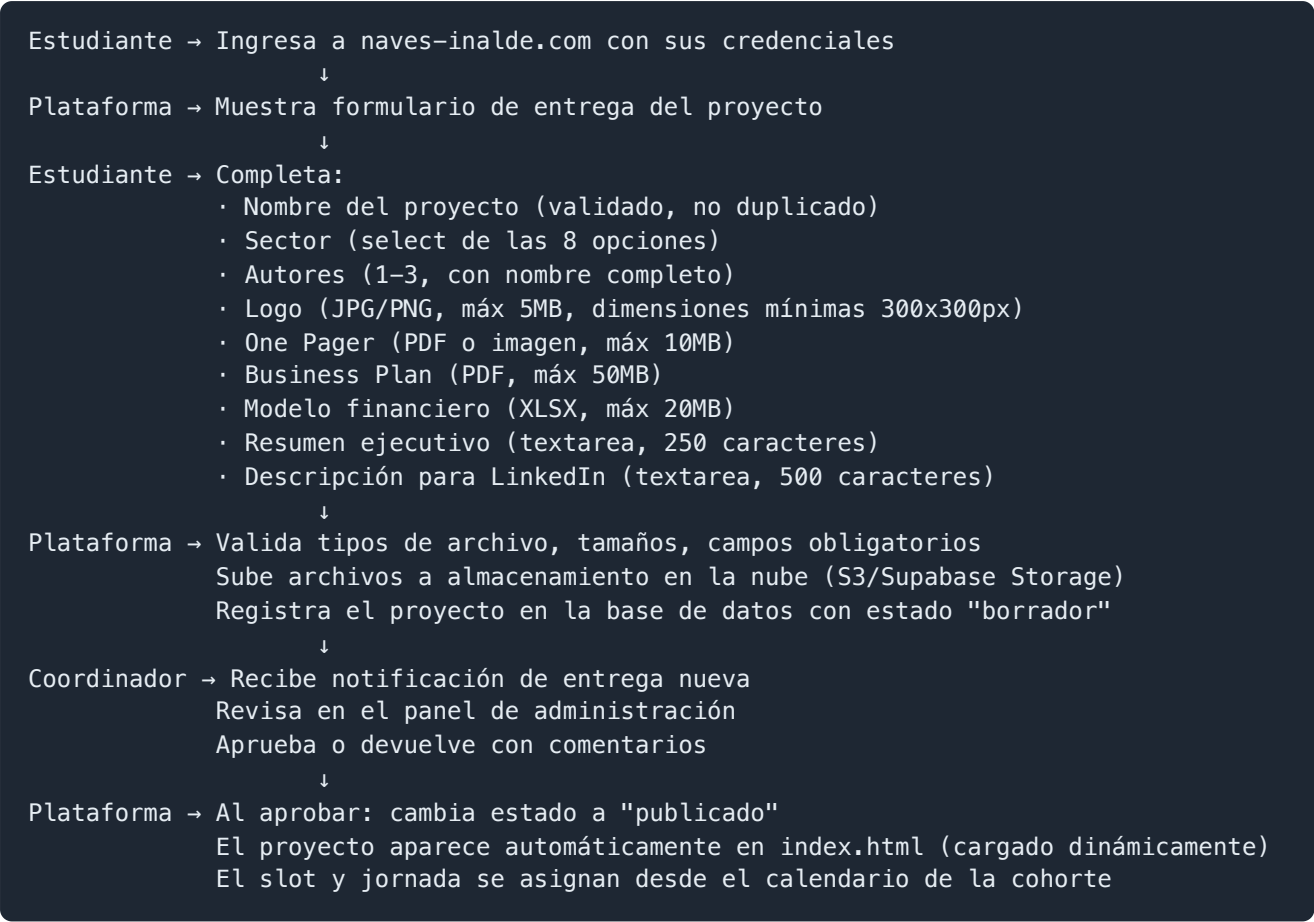
16.3 Nuevo proceso de ingesta de proyectos

16.3.1 Proceso actual (prototipo 2026)

```
Estudiante → Sube archivos a OneDrive (carpeta compartida)
              ↓
Coordinador → Ejecuta naves_generator.py manualmente
              ↓
Script → Lee carpetas OneDrive, extrae metadatos de nombres de archivo,
        copia archivos a NAVES_2026_Web/, genera index.html
              ↓
Coordinador → Reorganiza filas manualmente por horario de presentación
              ↓
Coordinador → netlify deploy --prod --dir .
```

Problemas: proceso manual, frágil, dependiente de que los estudiantes sigan exactamente la convención de nombres de archivo, y que el coordinador tenga Python y Netlify CLI instalados.

16.3.2 Proceso objetivo (versión profesional)



16.3.3 Especificaciones del formulario de entrega

Campos obligatorios:

Campo	Tipo	Validaciones
nombre_proyecto	text	Único en la cohorte, máx 60 chars, no puede quedar vacío
sector	select	Opciones definidas por el coordinador para esa cohorte
autores	array (1-3)	Nombre completo por cada autor; al menos 1
logo	file	JPG o PNG; máx 5 MB; mínimo 300x300 px
one_pager	file	PDF, JPG o PNG; máx 10 MB
resumen	textarea	Máx 250 caracteres (mostrar contador regresivo)
linkedin_post	textarea	Máx 600 caracteres

Campos opcionales:

Campo	Tipo	Notas
<code>business_plan</code>	file	PDF; máx 50 MB
<code>modelo_financiero</code>	file	XLSX o XLS; máx 20 MB
<code>video_pitch</code>	url	Link a YouTube o Vimeo
<code>web_proyecto</code>	url	Sitio web del emprendimiento (si existe)

Estado del proyecto: `borrador` → `en_revision` → `publicado` | `devuelto`

Cuando el coordinador devuelve un proyecto, debe poder escribir un comentario que le llega al estudiante por email.

16.3.4 Procesamiento de archivos en el servidor

Lo que hoy hace `naves_generator.py` localmente debe hacerse en el servidor al momento de la entrega:

- **Logo:** redimensionar a 400×400px máximo para miniatura (conservar el original también). Usar Sharp (Node.js) o Pillow (Python).
- **One pager PDF:** generar un thumbnail PNG de la primera página para preview rápido en la tabla. Usar pdf-thumbnail (Node) o pdf2image (Python).
- **Business plan PDF:** extraer texto de las primeras 4 páginas con pdfplumber (Python) o pdf-parse (Node) — para alimentar una futura generación automática del resumen con IA.
- **Modelo financiero XLSX:** no transformar, solo almacenar y exponer para descarga.
- **Todos los archivos:** generar URLs firmadas o públicas en el almacenamiento, guardar en la base de datos.

16.4 Modelo de datos para producción

Este es el esquema relacional recomendado. Está escrito en pseudoSQL — el desarrollador lo adaptará al ORM o lenguaje de migración que use.

```

-- — PROGRAMAS Y COHORTES —————

CREATE TABLE programa (
  id          SERIAL PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,    -- "Executive MBA"
  modalidad   VARCHAR(20) NOT NULL,     -- "Intensivo" | "Fin de Semana"
  codigo      VARCHAR(20) NOT NULL,     -- "EMBA-INT" | "EMBA-FS"
  institucion VARCHAR(100)              -- "INALDE Business School"
);

CREATE TABLE cohorte (
  id          SERIAL PRIMARY KEY,
  programa_id INT REFERENCES programa(id),
  nombre      VARCHAR(100) NOT NULL,    -- "FS 24-26"
  codigo      VARCHAR(20) NOT NULL,     -- "EMBA-FS-2026" (usado en URLs)
  anio_inicio INT,
  anio_fin    INT,
  estado      VARCHAR(20) DEFAULT 'activa', -- activa | archivada
  pwd_admin   VARCHAR(100),             -- hash bcrypt de la contraseña del panel admin
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- — CALENDARIO DE PRESENTACIONES —————

CREATE TABLE fecha_presentacion (
  id          SERIAL PRIMARY KEY,
  cohorte_id  INT REFERENCES cohorte(id),
  fecha       DATE NOT NULL,
  etiqueta    VARCHAR(50),             -- "Viernes 12 jun" – para mostrar en la UI
  orden       INT                      -- para ordenar: 1, 2, 3, 4
);

CREATE TABLE jornada (
  id          SERIAL PRIMARY KEY,
  fecha_presentacion_id INT REFERENCES fecha_presentacion(id),
  numero      INT NOT NULL,             -- 1, 2, 3... dentro del total de la cohorte
  hora_inicio TIME NOT NULL,            -- 13:50
  hora_fin    TIME NOT NULL,            -- 15:50
  duracion_slot_min INT DEFAULT 20,     -- minutos por proyecto
  pausa_entre_slots_min INT DEFAULT 5,
  salon       VARCHAR(100)              -- "Aula Magna", "Sala de Juntas B", etc.
);

-- — SECTORES (configurables por cohorte) —————

CREATE TABLE sector (
  id          SERIAL PRIMARY KEY,
  cohorte_id  INT REFERENCES cohorte(id),
  nombre      VARCHAR(100) NOT NULL,    -- "FinTech / Financiero"
  color_hex   VARCHAR(7) NOT NULL,      -- "#1a6b3c"
  icono       VARCHAR(10)               -- emoji o nombre de ícono
);

-- — PROYECTOS —————

CREATE TABLE proyecto (
  id          SERIAL PRIMARY KEY,

```

```

cohorte_id      INT REFERENCES cohorte(id),
sector_id       INT REFERENCES sector(id),
nombre          VARCHAR(100) NOT NULL,
resumen         TEXT,                -- máx 250 chars
linkedin_post   TEXT,                -- post para LinkedIn
estado          VARCHAR(20) DEFAULT 'borrador',
                -- borrador | en_revision | publicado | devuelto
es_confidencial BOOLEAN DEFAULT FALSE,
-- URLs de archivos en almacenamiento (S3 / Supabase Storage)
logo_url        TEXT,
logo_thumb_url  TEXT,                -- versión redimensionada para tabla
one_pager_url   TEXT,
one_pager_thumb_url TEXT,            -- PNG de la primera página
business_plan_url TEXT,
modelo_financiero_url TEXT,
video_pitch_url TEXT,
web_proyecto_url TEXT,
-- Metadatos de entrega
entregado_por   INT REFERENCES usuario(id),
entregado_at    TIMESTAMPTZ,
aprobado_por    INT REFERENCES usuario(id),
aprobado_at     TIMESTAMPTZ,
comentario_devolucion TEXT,
created_at      TIMESTAMPTZ DEFAULT NOW(),
updated_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE autor (
  id          SERIAL PRIMARY KEY,
  proyecto_id INT REFERENCES proyecto(id),
  nombre      VARCHAR(150) NOT NULL,
  email       VARCHAR(150),
  usuario_id  INT REFERENCES usuario(id), -- si tiene cuenta en la plataforma
  orden       INT DEFAULT 1               -- 1er, 2do, 3er autor
);

-- — SLOT DE PRESENTACIÓN (proyecto en jornada) —————

CREATE TABLE slot_presentacion (
  id          SERIAL PRIMARY KEY,
  jornada_id  INT REFERENCES jornada(id),
  proyecto_id INT REFERENCES proyecto(id),
  numero_slot INT NOT NULL,              -- posición dentro de la jornada: 1, 2, 3...
  hora_inicio TIME,                      -- calculada: hora_inicio_jornada + (n-1) * (duracion + pa
  hora_fin    TIME,
  UNIQUE (jornada_id, numero_slot),
  UNIQUE (jornada_id, proyecto_id)
);

-- — PANELISTAS —————

CREATE TABLE panelista (
  id          SERIAL PRIMARY KEY,
  cohorte_id  INT REFERENCES cohorte(id),
  nombre      VARCHAR(150) NOT NULL,
  email       VARCHAR(150) NOT NULL,
  empresa     VARCHAR(150),
  cargo       VARCHAR(150),

```



```
    bio          TEXT,
    foto_url     TEXT,
    UNIQUE (cohorta_id, email)
);
```

```
CREATE TABLE panelista_jornada (
    panelista_id INT REFERENCES panelista(id),
    jornada_id   INT REFERENCES jornada(id),
    PRIMARY KEY (panelista_id, jornada_id)
);
```

```
-- — LOGÍSTICA DE PANELISTAS —
```

```
CREATE TABLE logistica_panelista (
    id              SERIAL PRIMARY KEY,
    panelista_id    INT REFERENCES panelista(id),
    -- Transporte
    necesita_transporte BOOLEAN,
    direccion_recogida TEXT,
    hora_recogida    TIME,
    -- Por cada fecha de la cohorte puede necesitar o no transporte
    -- Almacenado como JSONB para flexibilidad: {"2026-06-12": true, "2026-06-13": false}
    transporte_por_fecha JSONB,
    -- Comidas
    -- JSONB: {"2026-06-12": true, "2026-06-19": false} (viernes = almuerzo)
    almuerzo_por_fecha JSONB,
    -- JSONB: {"2026-06-13": true, "2026-06-20": false} (sábados = desayuno)
    desayuno_por_fecha JSONB,
    -- Trazabilidad
    updated_at      TIMESTAMPTZ DEFAULT NOW(),
    updated_by      INT REFERENCES usuario(id)
);
```

```
-- — CONFIRMACIONES —
```

```
CREATE TABLE confirmacion_panelista (
    id              SERIAL PRIMARY KEY,
    panelista_id    INT REFERENCES panelista(id),
    confirmado      BOOLEAN DEFAULT FALSE,
    fecha_confirmacion TIMESTAMPTZ,
    email_enviado   BOOLEAN DEFAULT FALSE,
    email_enviado_at TIMESTAMPTZ,
    -- El panelista puede confirmar desde el link personalizado
    token_confirmacion VARCHAR(64) UNIQUE, -- UUID para el link personalizado
    ip_confirmacion   INET,
    created_at        TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- — USUARIOS DE LA PLATAFORMA —
```

```
CREATE TABLE usuario (
    id              SERIAL PRIMARY KEY,
    email           VARCHAR(150) UNIQUE NOT NULL,
    nombre          VARCHAR(150) NOT NULL,
    rol             VARCHAR(20) NOT NULL,
    -- 'super_admin' | 'coordinador' | 'panelista' | 'estudiante'
    cohorte_id      INT REFERENCES cohorte(id), -- NULL para super_admin
    auth_id         VARCHAR(100) UNIQUE,       -- ID de Supabase Auth / Auth0
);
```

```
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

Notas de diseño del modelo:

- `transporte_por_fecha`, `almuerzo_por_fecha`, `desayuno_por_fecha` son JSONB con clave = fecha ISO `"YYYY-MM-DD"` y valor boolean. Esto permite N fechas por cohorte sin cambiar el esquema.
- El campo `token_confirmacion` en `confirmacion_panelista` es un UUID único que se incluye en el link de invitación al panelista (<https://naves-inalde.com/panelista/confirmar?token=abc123>). Elimina la necesidad de que el panelista tenga cuenta — solo necesita el link.
- `slot_presentacion` tiene la hora calculable desde la jornada, pero se almacena explícitamente para facilitar consultas y para manejar casos especiales (slots con duración diferente, pausas largas para almuerzo, etc.).
- `token_participante` (definida en §16.16.5) es la tabla análoga para los estudiantes: un token UUID por proyecto que da acceso al portal de calendario. Un equipo de 3 autores comparte el mismo token — se envía el mismo link a los 3.

16.5 Roles y permisos

El sistema tiene 4 roles con permisos claramente delimitados:

Super Admin (INALDE central)

- Crear y configurar cohortes
- Definir sectores, fechas, jornadas y horarios por cohorte
- Gestionar coordinadores
- Acceder a datos de todas las cohortes
- Ver reportes históricos multi-cohorte

Coordinador (por cohorte)

- Gestionar panelistas de su cohorte (invitar, asignar jornadas)
- Revisar y aprobar/devolver entregas de proyectos
- Editar logística de panelistas
- Enviar emails de confirmación
- Ver y exportar resumen de logística
- Publicar/despublicar proyectos en el sitio público

Panelista

- Acceso solo a su cohorte, solo lectura de proyectos publicados
- Confirmar asistencia y declarar preferencias logísticas
- Ver la base de datos de proyectos de su jornada
- **No tiene cuenta** en la plataforma — accede por link con token

Estudiante / Autor

- Subir y actualizar los archivos de su propio proyecto
- Ver el estado de revisión de su entrega (borrador / en revisión / publicado / devuelto)
- Ver el comentario del coordinador si fue devuelto
- No puede ver proyectos de otros equipos hasta que la base de datos sea publicada

16.6 Integración con naves-inalde.com

16.6.1 Qué es naves-inalde.com

<https://naves-inalde.com/> es la plataforma central en desarrollo para gestionar el programa NAVES de INALDE. El sistema de panelistas y base de datos de proyectos documentado en este archivo es **uno de los módulos** de esa plataforma, no un producto independiente.

16.6.2 Módulos del sistema dentro de naves-inalde.com

naves-inalde.com/	
└─ /admin	Panel de administración (super admin + coordinadores)
└─ /cohortes	CRUD de cohortes
└─ /cohortes/{id}/	Dashboard de una cohorte
└─ proyectos/	Revisar y aprobar entregas
└─ panelistas/	Gestionar panelistas y logística
└─ calendario/	Configurar fechas, jornadas, slots
└─ reportes/	Resumen de logística + Excel de programación
└─ /configuracion	Sectores, templates de email, etc.
└─ /entrega	Portal del estudiante (autor)
└─ /login	Autenticación
└─ /mi-proyecto	Formulario de entrega y seguimiento de estado
└─ /panelista	Portal del panelista (acceso por token)
└─ /confirmar?token=X	Confirmación de asistencia + logística
└─ /proyectos?token=X	Base de datos de proyectos de su jornada
└─ /participante	Portal del participante/estudiante (acceso por token)
└─ ?token=X	Calendario completo de presentaciones – ver §16.16
└─ /naves/{cohorte-codigo}	Base de datos interna INALDE (equivalente a index.html)
└─ /proyectos	Posts LinkedIn, one pagers, programación del evento

16.6.3 Qué consume este módulo de la plataforma

- **Autenticación:** SSO desde naves-inalde.com. Si la plataforma usa Supabase Auth o Auth0, el panel de administración de panelistas debe usar el mismo proveedor de identidad. **No debe tener su propio sistema de login.**

- **Catálogo de proyectos:** Los proyectos se ingresan una sola vez en la plataforma (en `/entrega/mi-proyecto`). El panel de admin de panelistas y el sitio público los leen de la misma base de datos. No hay doble ingreso de datos.
- **Usuarios y roles:** La lista de coordinadores, estudiantes y panelistas viene de la plataforma central. El módulo de panelistas no gestiona usuarios independientemente.
- **Configuración de la cohorte:** Las fechas y jornadas se configuran en la plataforma central (`/admin/cohortes/{id}/calendario`). El módulo de panelistas las consume vía API.

16.6.4 Qué expone este módulo hacia la plataforma

- **API REST** (o endpoints de Supabase directos) para:
 - `GET /api/cohortes/{id}/proyectos` — lista pública de proyectos publicados
 - `GET /api/cohortes/{id}/panelistas` — lista para el coordinador
 - `GET /api/cohortes/{id}/logistica` — datos de logística para exportación
 - `PUT /api/panelistas/{id}/logistica` — actualizar logística desde el admin
 - `POST /api/panelistas/{id}/confirmacion` — registrar confirmación desde el portal del panelista
 - `POST /api/emails/enviar-confirmacion` — trigger de envío de email
- **Webhooks (opcional):**
 - Cuando un panelista confirma → notificar al coordinador
 - Cuando un proyecto es aprobado → notificar al equipo de estudiantes

16.6.5 Migración del prototipo a la plataforma

El prototipo actual (`panelistas.html`, `admin.html`, `index.html`) puede coexistir con la plataforma en construcción. La estrategia recomendada es:

Fase de transición:

1. La plataforma desarrolla el módulo de entrega de proyectos (`/entrega/mi-proyecto`) — los estudiantes ingresan sus datos ahí.
2. La plataforma expone una API `GET /api/cohortes/{id}/proyectos` que retorna los proyectos publicados.
3. `index.html` se modifica para cargar los proyectos dinámicamente desde esa API en lugar de tenerlos hardcodeados. El resto de la página no cambia.
4. `admin.html` y `panelistas.html` se mantienen como están hasta que la plataforma tenga los módulos equivalentes listos.

Fase de reemplazo:

5. La plataforma lanza `/admin/cohortes/{id}/panelistas` — reemplaza `admin.html`.
6. La plataforma lanza `/panelista/confirmar?token=X` — reemplaza `panelistas.html`.
7. La plataforma lanza `/naves/{codigo}/proyectos` — reemplaza `index.html`.
8. El sitio Netlify (`navesfs.netlify.app`) se da de baja o redirige a `naves-inalde.com`.

16.6.6 Formato real de la API de la plataforma (verificado el 15 jun 2026)

La plataforma `naves-inalde.com` **ya existe y está en construcción**. Se verificó en vivo lo siguiente (inspeccionando las peticiones del panel de admin):

- **Backend:** Supabase (el JWT de sesión lo emite `...supabase.co`). Esto **confirma** el stack recomendado en §16.7.
- **Autenticación:** JWT Bearer de Supabase Auth en el header `Authorization` . El token incluye `app_metadata.app_role` (ej. `super_admin`), `email` , `profesor_id` . Los tokens son de corta duración (~1 h). → La integración debe usar este mismo proveedor (SSO), sin login propio.
- **Endpoint de cohortes:** `GET https://naves-inalde.com/api/admin/cohortes`

Formato de respuesta (array de cohortes):

```
[
  { "id": "fs-24-26", "etiqueta": "MBA FS 24-26", "fecha_inicio": "2024-01-18", "fecha_
  { "id": "fs-26-28", "etiqueta": "MBA FS 26-28", "fecha_inicio": "2026-01-17", "fecha_
  { "id": "int-24-26", "etiqueta": "MBA INT 24-26", "fecha_inicio": "2024-01-13", "fecha_
]
```

Campo	Significado	Mapea a (§16.4)
<code>id</code>	Código de cohorte (<code>fs-24-26</code> , <code>int-26-28</code>)	<code>cohorte.codigo</code>
<code>etiqueta</code>	Nombre legible (<code>MBA FS 24-26</code>)	<code>cohorte.nombre</code>
<code>fecha_inicio</code> / <code>fecha_fin</code>	Fechas del programa (no de las presentaciones)	<code>cohorte.anio_inicio/fin</code>

⚠ Importante: `fecha_inicio` / `fecha_fin` son las del programa académico, **no las fechas de presentación del NAVES**. Las **fechas de presentación** y los **proyectos** viven en el detalle de cada cohorte (endpoints aún por confirmar — al momento de la verificación no había ninguna cohorte con presentaciones configuradas; presumiblemente `GET /api/admin/cohortes/{id}` o sub-rutas tipo `/proyectos` y `/presentaciones`).

Convención de `id` observada: prefijo `fs-` (Fin de Semana) o `int-` (Intensivo) — coincide con el `tipo` del programador (§9.4).

Implicación para el programador (§9.3): a futuro, tanto la **lista de proyectos** (hoy `PROYECTOS_BASE`) como las **fechas de presentación** (hoy `DEFAULTS`) deben **recibirse de esta API** en lugar de estar precargadas. El importador JSON (§9.6b) es el puente: cuando se conozca el formato exacto de proyectos y fechas del detalle de cohorte, se mapea a la entrada `{ proyectos:[...], fechas:[...] }` que consume el programador. El profesor sigue decidiendo solo el orden y los horarios.

16.7 Stack tecnológico recomendado

Esta es una recomendación, no un requerimiento. El desarrollador debe evaluarla en el contexto de lo que ya usa naves-inalde.com.

Opción A — Full Supabase (recomendada si naves-inalde.com no tiene stack definido)

Capa	Tecnología	Justificación
Frontend	Next.js 14+ (App Router)	SSR/SSG para el sitio público; CSR para el admin
Base de datos	Supabase (PostgreSQL)	Incluye Auth, Storage, Realtime, REST API auto-generada
Autenticación	Supabase Auth	Soporta email+password, Magic Link, OAuth (Google, Microsoft)
Almacenamiento de archivos	Supabase Storage	Buckets para logos, PDFs, XLS; URLs firmadas
Email	Resend (resend.com)	API moderna, soporte React Email templates, 100 emails/día gratis
Hosting	Vercel	Integración nativa con Next.js; preview deployments por PR
Generación de thumbnails	Sharp (Node.js) en Vercel Edge Functions	Redimensionar logos, generar preview de PDFs

Costo estimado en producción (por cohorte, ~2 meses de uso intensivo):

- Supabase Free tier: suficiente para este volumen (500MB DB, 1GB Storage, 50K auth users)
- Vercel Hobby: gratuito para proyectos no comerciales
- Resend: gratuito hasta 100 emails/día (más que suficiente)
- **Total: \$0/mes en producción para el volumen de NAVES**

Opción B — Si naves-inalde.com ya usa otro stack

Si la plataforma tiene backend propio (Laravel, Django, Rails, Express, etc.), lo correcto es:

- Usar ese mismo backend para los nuevos módulos
- Usar la misma base de datos ya existente (agregar las tablas del §16.4)
- Usar el mismo proveedor de autenticación ya integrado
- No crear un silo tecnológico separado

Lo que NO debe hacer el desarrollador: crear una segunda aplicación independiente con su propia base de datos que luego hay que sincronizar con la plataforma principal. El objetivo es un solo sistema integrado.

16.8 API de confirmación de panelistas: rediseño con tokens

El sistema actual de confirmación tiene dos problemas:

1. Los panelistas acceden al formulario de logística sin ningún mecanismo de identificación — cualquiera que tenga el link puede ingresar datos con cualquier email.
2. Las confirmaciones de Netlify Forms y los datos de logística de JSONbin están en dos sistemas diferentes, sin relación entre sí.

Flujo rediseñado con tokens

```
Coordinador asigna panelistas a jornadas en admin
↓
Sistema genera token UUID único por panelista por cohorte
↓
Sistema envía email: "Confirma tu asistencia → https://naves-inalde.com/panelista/confirm
↓
Panelista hace clic en el link
↓
Plataforma valida el token:
  · Busca confirmacion_panelista WHERE token_confirmacion = 'abc123xyz'
  · Si no existe o ya fue usado: muestra error "Link inválido o expirado"
  · Si existe: carga el contexto del panelista (nombre, jornadas asignadas)
↓
Panelista ve el formulario pre-cargado con sus datos y jornadas específicas
↓
Panelista completa logística y confirma
↓
Plataforma:
  · Actualiza logistica_panelista con los datos
  · Actualiza confirmacion_panelista: confirmado=true, fecha_confirmacion=NOW(), ip=X
  · Envía email de confirmación al panelista (resumen de lo que ingresó)
  · Envía notificación al coordinador
↓
El token puede invalidarse después de N días o dejarse activo para editar
```

Ventajas:

- Cada panelista solo puede modificar sus propios datos
- El coordinador ve exactamente quién confirmó y cuándo
- No hay necesidad de que el panelista cree cuenta ni recuerde contraseña
- Se puede reenviar el link en cualquier momento si el panelista perdió el email
- El token puede incluir un TTL (ej: expira 7 días antes de la primera jornada)

16.9 Generación automática de contenido con IA (resúmenes y posts)

Estado actual

Los resúmenes de 250 caracteres y los posts de LinkedIn están hardcodedos en `naves_generator.py` → `SUMMARIES_DATA`. Fueron generados una vez con Claude API y luego editados manualmente.

Flujo automatizado para versiones futuras

Cuando un estudiante sube el Business Plan (PDF) y el One Pager, el servidor debe:

1. Extraer texto de las primeras 4 páginas del Business Plan con `pdfplumber` (Python) o `pdf-parse` (Node).
2. Llamar a la Claude API con un prompt estructurado:

Eres el comunicador de INALDE Business School para el programa NAVES.
Analiza el siguiente texto del Business Plan y genera:

1. RESUMEN (máx 250 caracteres, incluye producto, mercado y diferenciador):
2. POST LINKEDIN (máx 600 caracteres, incluye nombres de autores, nombre del proyecto, descripción breve, frase de cierre inspiradora, y los hashtags:
#SoyINALDE #NavesINALDE #ExecutiveMBA #Líder #INALDE #Liderazgo #MBA #EMBA #NAVES):

Texto del Business Plan:
{texto_extraido}

Nombres de los autores: {autores}
Nombre del proyecto: {nombre_proyecto}

3. Guardar el resumen y post generados en la base de datos como borrador (`estado='borrador_ia'`).
4. El coordinador puede editar el texto antes de aprobarlo.
5. Al aprobar el proyecto, el resumen y post quedan `estado='aprobado'` y se publican.

Modelo recomendado: `claude-haiku-4-5` (rápido y económico; adecuado para resúmenes cortos).
Costo estimado: <\$0.01 por proyecto.

Modelo API:

```
// Endpoint: POST /api/proyectos/{id}/generar-contenido
const response = await anthropic.messages.create({
  model: "claude-haiku-4-5-20251001",
  max_tokens: 400,
  messages: [{
    role: "user",
    content: buildPrompt(bpText, autores, nombreProyecto)
  }]
});
```

16.10 Gestión del calendario de presentaciones

Problema actual

El horario de presentaciones (qué proyecto va en qué slot de qué jornada) existe **solo** como el orden manual de filas `<tr>` en `index.html` . No hay ninguna estructura de datos detrás.

Solución: módulo de calendario en el admin

El coordinador debería poder configurar el calendario desde una interfaz, no editando HTML:

Vista de configuración del calendario:

Cohorte FS 24-26 – Calendario de presentaciones

Fecha: Viernes 12 de junio de 2026

Jornada 1 – 13:50 a 15:50

Slot 1 (13:50–14:10): [Bevo ▼] [x]

Slot 2 (14:15–14:35): [SATORI ▼] [x]

Slot 3 (14:40–15:00): [AKOS ▼] [x]

Slot 4 (15:05–15:25): [FioYa ▼] [x]

Slot 5 (15:30–15:50): [VecinoPro ▼] [x]

[+ Agregar slot]

Jornada 2 – 16:30 a 18:30

Slot 1 (16:30–16:50): [ARCO ▼] [x]

...

Con drag-and-drop, el coordinador puede reordenar proyectos entre slots. Los horarios se recalculan automáticamente.

Lógica de cálculo de slots:

$$\text{hora_slot_n} = \text{hora_inicio_jornada} + (n - 1) \times (\text{duracion_slot_min} + \text{pausa_entre_slots_min})$$

Ej: jornada empieza a las 13:50, slots de 20 min, pausas de 5 min:

- Slot 1: 13:50
- Slot 2: 14:15 (13:50 + 25 min)
- Slot 3: 14:40 (14:15 + 25 min)

16.11 Gestión del Excel de logística: mejoras para producción

El CSV que genera actualmente `descargarExcel()` en `admin.html` es funcional pero tiene limitaciones. En la versión profesional:

Exportaciones necesarias:

Nombre	Contenido	Formato	Quién lo usa
<code>logistica_general.xlsx</code>	Una fila por panelista x jornada con transporte, comidas	XLSX con formato	Coordinador INALDE
<code>transporte_por_fecha.xlsx</code>	Una hoja por fecha; lista de recogidas con dirección y hora	XLSX	Servicio de transporte externo
<code>comidas_por_jornada.xlsx</code>	Conteo de almuerzos y desayunos por fecha	XLSX	Proveedor de catering
<code>confirmaciones.xlsx</code>	Estado de confirmación de cada panelista	XLSX	Coordinador académico

Todos deben generarse en el servidor (no en el navegador) para evitar el límite de SheetJS en grandes volúmenes, y enviarse como descarga directa desde la API (`Content-Disposition: attachment`).

16.12 Notificaciones y comunicaciones

Estado actual

- **Email de confirmación al panelista:** EmailJS desde el navegador, template fijo en EmailJS dashboard.
- **No hay notificaciones al coordinador** cuando un panelista confirma.
- **No hay recordatorios automáticos** a panelistas que no han confirmado.

Sistema de comunicaciones objetivo

Evento	Destinatario	Canal	Contenido
Proyecto entregado	Coordinador	Email + notificación en plataforma	"El equipo X entregó su proyecto"
Proyecto devuelto	Autores del proyecto	Email	"Tu proyecto fue devuelto: [comentario]"
Proyecto aprobado	Autores del proyecto	Email	"Tu proyecto fue publicado en la base de datos"
Invitación a panelista	Panelista	Email personalizado	Link de confirmación + info de jornadas
Panelista confirma	Coordinador	Email	"Francisco Forero confirmó asistencia"
Recordatorio (T-7 días)	Panelistas no confirmados	Email automático	"Faltan 7 días — confirma tu asistencia"
Resumen logístico (T-2 días)	Panelistas confirmados	Email	Su resumen de jornadas + transporte + comidas

Implementación: Usar cron jobs en el servidor (o Supabase Edge Functions con `pg_cron`) para los emails automáticos de recordatorio y resumen.

Template de email: Un solo template React Email reutilizable que recibe props (`tipo: 'invitacion' | 'recordatorio' | 'resumen'` , `datos: PanelistaData`) y renderiza el HTML del email. Esto centraliza el branding del email.

16.13 Sitio público multi-cohorte: `index.html` → `/naves/{cohorte}`

El `index.html` actual es una página estática con todos los datos de los 34 proyectos embebidos. La versión profesional debe ser una página dinámica dentro de `naves-inalde.com`.

URL structure

<code>naves-inalde.com/naves/emba-fs-2026/proyectos</code>	→ Cohorte 2026
<code>naves-inalde.com/naves/emba-fs-2028/proyectos</code>	→ Cohorte 2028 (futura)
<code>naves-inalde.com/naves/</code>	→ Redirige a la cohorte activa

Carga de datos

```
// Next.js – app/naves/[cohortel]/proyectos/page.tsx
export async function generateStaticParams() {
  const cohortes = await getCohortes(); // solo las activas/publicadas
  return cohortes.map(c => ({ cohorte: c.codigo }));
}

export default async function ProyectosPage({ params }) {
  const proyectos = await getProyectosPublicados(params.cohorte);
  const calendario = await getCalendario(params.cohorte);
  return <TablaProyectos proyectos={proyectos} calendario={calendario} />;
}
```

Con Next.js ISR (Incremental Static Regeneration), la página se regenera automáticamente cada N minutos cuando el coordinador aprueba un proyecto nuevo. No hay redeploy manual.

Acceso controlado (si se decide proteger)

Actualmente `index.html` no tiene contraseña. En versiones futuras podría requerir login con cuenta INALDE para acceder a los one paggers y modelos financieros (que son documentos estratégicos). Los posts de LinkedIn y los resúmenes de 250 chars podrían mantenerse públicos.

16.14 Checklist de tareas para el desarrollador profesional

Esta es la lista ordenada de trabajo para construir la versión 2.0 del sistema, por fases:

Fase 0 — Prerequisitos (antes de empezar a codificar)

- [] Confirmar el stack tecnológico de naves-inalde.com (¿ya tiene backend? ¿qué framework? ¿qué base de datos?)
- [] Confirmar el proveedor de autenticación ya usado (Google Workspace INALDE, Microsoft 365, Auth0, Supabase Auth, otro)
- [] Confirmar si hay un repositorio git existente para naves-inalde.com o si se parte de cero
- [] Confirmar el proveedor de almacenamiento de archivos (AWS S3, Google Cloud Storage, Supabase Storage, Azure Blob)
- [] Revisar si hay restricciones de datos (GDPR, datos colombianos de protección de datos personales — Ley 1581)

Fase 1 — Base de datos y autenticación

- [] Crear el schema de base de datos del §16.4
- [] Configurar autenticación con los roles del §16.5
- [] Implementar CRUD de cohortes, fechas, jornadas y sectores
- [] Implementar CRUD de proyectos (sin formulario de entrega aún)
- [] Importar los datos de la cohorte 2026 (los 34 proyectos + 12 panelistas + calendario) como seed de la base de datos

Fase 2 — Portal del panelista (equivalente a `panelistas.html` + `admin.html`)

- [] Implementar generación y envío de tokens de confirmación
- [] Implementar `/panelista/confirmar?token=X` — formulario de logística con fechas dinámicas desde la BD
- [] Implementar `/admin/cohortes/{id}/panelistas` — tabla con stats, logística editable por modal, envío de emails
- [] Implementar generación del resumen de logística y exportación Excel
- [] Implementar envío de emails de confirmación con Resend (reemplazando EmailJS)

Fase 3 — Portal de entrega de proyectos

- [] Implementar `/entrega/mi-proyecto` — formulario del §16.3.3
- [] Implementar subida y procesamiento de archivos (resize de logos, thumbnail de PDFs)
- [] Implementar flujo de revisión coordinador (aprobar / devolver con comentario)
- [] Implementar generación automática de resumen y post LinkedIn con Claude API (§16.9)

Fase 4 — Base de datos interna y portal de participantes

- [] Implementar `/naves/{cohorte}/proyectos` — carga dinámica desde BD (equivalente a `index.html`)
- [] Implementar filtros por día y búsqueda (equivalente al JS actual de `filterTable()`)
- [] Implementar **dos botones de exportación Excel**: (a) el actual de contenido editorial (posts, resúmenes, logos) y (b) el nuevo de **programación del evento** con columnas fecha, jornada, slot, hora inicio, hora fin, proyecto, autores (ver especificación §6.8)
- [] Configurar ISR para que la página se regenere al aprobar proyectos
- [] Implementar portal de participantes `/participante?token=X` (ver §16.16 completo):
- [] Tabla `token_participante` — generación y envío de tokens por proyecto
- [] Endpoint `GET /api/participante/calendario?token=X`
- [] UI: sección "Tu presentación" con datos del proyecto + slot resaltado
- [] UI: tabla del calendario completo con fila propia resaltada y etiqueta "← tu presentación"
- [] Filtro por día dentro del calendario completo
- [] Generación de archivo `.ics` para "Agregar a mi calendario"

Fase 5 — Módulo de calendario

- [] Implementar configuración de fechas/jornadas/slots con UI drag-and-drop
- [] Implementar asignación de proyectos a slots
- [] Validar que no haya proyectos sin slot asignado al publicar

Fase 6 — Comunicaciones automáticas

- [] Implementar templates de email con React Email
- [] Implementar cron de recordatorios (T-7 días a no confirmados)
- [] Implementar email de resumen logístico (T-2 días)
- [] Implementar notificaciones en plataforma (campana de notificaciones)

16.16 Sitio de participantes — Portal de calendario para estudiantes

16.16.1 Propósito

Los estudiantes (autores de proyectos) necesitan saber cuándo deben presentar: en qué fecha, en qué jornada y a qué hora les toca. Este portal les da esa información de forma clara, con el contexto del calendario completo de la cohorte.

Es un sitio **separado** de `panelistas.html` (que es solo para los evaluadores) y de `index.html` (que es para el equipo interno de INALDE). El estudiante no necesita ver los posts de LinkedIn, ni los botones de descarga de otros proyectos, ni la logística de panelistas.

16.16.2 Mecanismo de acceso

Igual que el portal de panelistas: **acceso por link personalizado con token UUID**, sin login ni contraseña.

```
https://naves-inalde.com/participante?token=def456uvw
```

El token identifica al equipo (no al individuo dentro del equipo). Un solo token por proyecto, compartido con todos los autores del equipo.

Generación del token: El coordinador genera los tokens desde el panel de administración, igual que con los panelistas. El sistema envía el link por email a todos los autores del proyecto.

Validez: El token es válido desde que se genera hasta N días después del evento (para que los estudiantes puedan consultar el horario antes y durante las jornadas). No expira antes del evento.

Una vez dentro: El participante ve el **calendario completo** de todas las presentaciones de la cohorte — no solo la suya. Puede ver en qué slot presenta su equipo y también el orden de los demás proyectos.

16.16.3 Contenido de la página

La página tiene dos secciones:

Sección 1 — "Tu presentación" (destacada, visible sin scroll)

Muestra con énfasis visual la información del proyecto del token:

TU PRESENTACIÓN

 Viernes 12 de junio de 2026

 Jornada 1 · 13:50 – 15:50

 Slot 3 – Hora de inicio: 14:40

Proyecto: AKOS

Autores: Héctor Rodrigo Arias Cueca,
Cesar Julián Pérez Garavito

AKOS es una FinTech colombiana que financia medicamentos y procedimientos urgentes no cubiertos por salud. Aprobación en menos de 5 minutos, desembolso directo al proveedor.

[Agregar a mi calendario]

Sección 2 – "Calendario completo"

Tabla con todas las presentaciones de la cohorte, con la fila del proyecto del participante resaltada:

Slot	Hora	Proyecto	Autores	Resumen breve
1	13:50	Bevo	Silvio Terán...	Plataforma de bienestar...
2	14:15	SATORI	Rina Molina	Empleabilidad senior 50+...
3	14:40	AKOS ← tú	Héctor Arias...	FinTech para salud...
4	15:05	FioYa	Pablo Molina	Digitaliza el fiado...

La fila del equipo está resaltada con un fondo de color y una etiqueta "← tu presentación".

Filtros disponibles: por día (igual que en [index.html](#)).

16.16.4 Lo que NO aparece en este sitio

- Posts de LinkedIn (son para comunicaciones, no para los autores)
- Botones de descarga de logos o one paggers
- Información de logística de panelistas
- Datos de contacto de otros participantes o panelistas
- Cualquier dato de otros proyectos más allá de nombre, autores y resumen breve

16.16.5 Campos necesarios en la base de datos

Este portal no requiere tablas nuevas. Usa las ya definidas en §16.4:

- `proyecto` → nombre, resumen, autores
- `slot_presentacion` → numero_slot, hora_inicio, hora_fin
- `jornada` → numero, hora_inicio, hora_fin
- `fecha_presentacion` → fecha, etiqueta

Requiere una tabla adicional para los tokens de acceso de participantes:

```
CREATE TABLE token_participante (  
  id          SERIAL PRIMARY KEY,  
  proyecto_id INT REFERENCES proyecto(id),  
  token       VARCHAR(64) UNIQUE NOT NULL, -- UUID  
  generado_at TIMESTAMPTZ DEFAULT NOW(),  
  generado_por INT REFERENCES usuario(id),  
  -- Sin expiración anticipada al evento; expira N días después  
  expira_at   TIMESTAMPTZ  
);
```

16.16.6 API necesaria

```
GET /api/participante/calendario?token=def456uvw
```

Respuesta:

```
{  
  "mi_proyecto": {  
    "id": 3,  
    "nombre": "AKOS",  
    "resumen": "FinTech colombiana...",  
    "autores": ["Héctor Rodrigo Arias Cueca", "Cesar Julián Pérez Garavito"],  
    "slot": {  
      "numero": 3,  
      "hora_inicio": "14:40",  
      "hora_fin": "15:00",  
      "jornada": { "numero": 1, "hora_inicio": "13:50", "hora_fin": "15:50" },  
      "fecha": { "fecha": "2026-06-12", "etiqueta": "Viernes 12 jun" }  
    }  
  },  
  "calendario": [  
    {  
      "fecha_etiqueta": "Viernes 12 jun",  
      "jornadas": [  
        {  
          "numero": 1,  
          "hora_inicio": "13:50",  
          "hora_fin": "15:50",  
          "slots": [  
            { "numero": 1, "hora_inicio": "13:50", "proyecto": "Bevo", "autores": "...",  
              { "numero": 2, "hora_inicio": "14:15", "proyecto": "SATORI", ... },  
            { "numero": 3, "hora_inicio": "14:40", "proyecto": "AKOS", "es_mio": true, ..  
            ...  
          ]  
        }  
      ]  
    },  
    ...  
  ]  
}
```

El campo `"es_mio": true` en el slot permite que el frontend resalte esa fila sin lógica adicional.

16.16.7 Botón "Agregar a mi calendario"

La sección "Tu presentación" debe tener un botón que genere un archivo `.ics` para que el estudiante agregue su slot a Google Calendar, Outlook o Apple Calendar.

```
function generarICS(slot) {
  const dtstart = slot.fecha.replace(/-/g, '') + 'T' + slot.hora_inicio.replace(':', '') +
  const dtend   = slot.fecha.replace(/-/g, '') + 'T' + slot.hora_fin.replace(':', '') + '00
  const ics = [
    'BEGIN:VCALENDAR',
    'VERSION:2.0',
    'BEGIN:VEVENT',
    `DTSTART:${dtstart}`,
    `DTEND:${dtend}`,
    `SUMMARY:Presentación NAVES - ${slot.proyecto}`,
    `DESCRIPTION:Slot ${slot.numero} · Jornada ${slot.jornada_numero}\\nINALDE Business S
    `LOCATION:INALDE Business School`,
    'END:VEVENT',
    'END:VCALENDAR'
  ].join('\r\n');
  const blob = new Blob([ics], { type: 'text/calendar' });
  const a = document.createElement('a');
  a.href = URL.createObjectURL(blob);
  a.download = `NAVES_2026_${slot.proyecto}.ics`;
  a.click();
}
```


16.16.8 Comparación con los otros tres sitios del sistema

Característica	<code>panelistas.html</code>	<code>index.html</code>	Portal participante
Audiencia	Evaluadores externos	Equipo interno INALDE	Estudiantes / autores
Acceso	Link abierto (por nombre del panelista)	Sin restricción actual	Token por equipo
Ve proyectos de otros	Sí (todos los de su jornada)	Sí (todos)	Sí (todos, solo nombre + resumen)
Ve su propia info destacada	Sí (sus jornadas)	N/A	Sí (su slot resaltado)
Posts LinkedIn	No	Sí	No
Descargas	Sí (one pagers)	Sí (logos + one pagers)	No
Logística de panelistas	Sí (la propia)	No	No
Exportar Excel	No	Sí (comunicaciones)	No (solo agregar a calendario)
Confirmación / acción	Sí (confirma asistencia)	No	No (solo lectura)

16.15 Lo que no debe reinventarse

Hay partes del prototipo que funcionan bien y deben mantenerse conceptualmente, aunque la implementación cambie:

Concepto	Prototipo	Versión profesional
Logística por fecha (arrays)	JSONB en JSONbin	JSONB en PostgreSQL — mismo concepto
Token de acceso sin login	No existe (se usa email manual)	UUID por panelista — implementar nuevo
Resumen de logística por jornada	Texto plano generado en JS	Mismo formato, generado en servidor
Distinción viernes/sábado para comidas	<code>tieneViernes()</code> / <code>tieneSabado()</code>	Calculado desde las fechas en la BD
Feedback visual de "Copiado"	2 segundos en botón	Mantener mismo UX
Diseño visual INALDE/NAVES	Tokens CSS en las 3 páginas	Design system compartido (variables CSS o Tailwind theme)
Exportación CSV/XLSX separador <code>;</code>	En <code>descargarExcel()</code>	Mantener <code>;</code> como separador para Excel en español

17. Guía de migración a Supabase

Guía concreta para el desarrollador (que **ya usa Supabase** en naves-inalde.com). El objetivo: mover NAVES de las páginas estáticas + JSONbin + contraseña fija a un backend real, y desbloquear la lectura del calendario con Opus (que en Netlify de bajo plan no cupo — §10). El esquema de tablas completo está en §16.4; aquí va el "cómo conectar".

17.1 Qué reemplaza a qué

Hoy (prototipo)	En Supabase
JSONbin de logística (§4.1)	Tabla <code>logistica_panelista</code> + <code>confirmacion_panelista</code> (§16.4)
JSONbin de programación (§4.2)	Tablas <code>slot_presentacion</code> / <code>jornada</code> / <code>fecha_presentacion</code> (§16.4)
JSONbin de extracción de calendario (§10)	Edge Function <code>extraer-calendario</code> (abajo)
Contraseña fija <code>Mba2026</code> (admin/programador)	Supabase Auth + rol en el JWT (<code>app_metadata.app_role</code>)
Proyectos/fechas hardcodedos (<code>PROYECTOS_BASE</code> , <code>DEFAULTS</code>)	Tablas <code>proyecto</code> , <code>fecha_presentacion</code>
Logos y PDFs en carpetas <code>logos/</code> , <code>pdfs/</code>	Supabase Storage (buckets <code>logos</code> , <code>documentos</code>)
EmailJS / Netlify Forms (panelistas)	Insert directo en tabla + Edge Function que manda el correo (Resend)

17.2 Conexión desde el front-end

Datos públicos del panel de Supabase (Project Settings → API): **Project URL** y **anon key**. La anon key es segura en el cliente porque el acceso lo gobierna **RLS** (§17.5).

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
<script>
  const sb = supabase.createClient('https://<PROJECT>.supabase.co', '<ANON_KEY>');

  // Lectura (reemplaza el fetch a JSONbin):
  const { data: proyectos } = await sb.from('proyecto')
    .select('*').eq('cohorte_id', cohorteId).eq('estado', 'publicado');

  // Escritura protegida (requiere sesión iniciada):
  await sb.from('logistica_panelista').update({ transporte_por_fecha: {...} })
    .eq('panelista_id', id);
</script>
```

17.3 Autenticación (reemplaza la contraseña **Mba2026**)

naves-inalde.com ya emite JWT de Supabase Auth con **app_metadata.app_role** (visto en vivo: **super_admin**). Las páginas admin/programador deben usar ese login en vez de la contraseña fija:

```
await sb.auth.signInWithPassword({ email, password }); // o el SSO ya existente
const { data: { user } } = await sb.auth.getUser(); // user.app_metadata.app_role
```

Las páginas públicas (index para internos, portal de panelistas por token) no necesitan login; las protege RLS y/o tokens (§16.8, §16.16).

17.4 Edge Function para leer el calendario (el paso que se bloqueó en Netlify)

Las Edge Functions corren en servidor sin el límite de ~26 s de Netlify, así que **sí permiten Opus**. La **ANTHROPIC_API_KEY** va como secreto, nunca en el cliente.

```
// supabase/functions/extraer-calendario/index.ts (Deno)
const cors = { "Access-Control-Allow-Origin": "*", "Access-Control-Allow-Headers": "autho

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: cors });
  const { pdf_base64, tipo } = await req.json();
  const apiKey = Deno.env.get("ANTHROPIC_API_KEY");

  const schema = { /* mismo esquema de §10: tipo_detectado, anio, jornadas_clase,
                    festivis, semana_santa_lunes, presentaciones, grado, eventos, notas
  const prompt = `Lee el calendario académico del MBA de INALDE ... (mismo prompt de §10,
                  con énfasis en verificar el AÑO de cada celda). ${tipo ? "Modalidad: "+

  const resp = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "x-api-key": apiKey, "anthropic-version": "2023-06-01", "content-type": "a
    body: JSON.stringify({
      model: "claude-opus-4-8", // el modelo bueno; sin límite de tiempo aquí
      max_tokens: 16000,
      thinking: { type: "adaptive" },
      output_config: { format: { type: "json_schema", schema }, effort: "high" },
      messages: [{ role: "user", content: [
        { type: "document", source: { type: "base64", media_type: "application/pdf", data
        { type: "text", text: prompt }
      ]}]
    })
  });
  const data = await resp.json();
  const text = (data.content || []).find((b: any) => b.type === "text")?.text ?? "{}";
  return new Response(text, { headers: { ...cors, "content-type": "application/json" } });
});
```

Despliegue y secreto:

```
supabase functions deploy extraer-calendario
supabase secrets set ANTHROPIC_API_KEY=sk-ant-...
```

Llamada desde el front-end (sin polling — la función responde cuando Opus termina):

```
const { data, error } = await sb.functions.invoke('extraer-calendario', { body: { pdf_bas
```

El prototipo de Netlify ([netlify/functions/extraer-calendario-background.js](#) , §10) sirve de referencia 1:1 para el prompt y el esquema — solo cambia el runtime (Deno en vez de Node) y desaparecen el JSONbin de resultados y el polling, porque la Edge Function responde de forma síncrona.

17.4b Edge Function `generar-descripciones` (resúmenes y posts desde el one pager)

Por qué existe: ver el incidente del 16 jun 2026 en §5.4 (Paso 5). Los resúmenes y posts de LinkedIn **dejaron de hardcodearse**: ahora se generan con IA leyendo el one pager real de cada proyecto, sea cual sea su formato. Esta función materializa ese requisito.

Flujo:

1. El proyecto sube su one pager al Storage (bucket `onepagers` , §17.6).
2. La Edge Function recibe el archivo (o su path en Storage) + los **autores** del proyecto.
3. Lee el contenido **sea cual sea el formato**:
 - PDF con texto → se manda como `document` a Claude.
 - PDF imagen / PNG / JPG → se manda como `image` (Claude lo lee con **visión**). No hace falta OCR aparte; Opus lee one pagers densos directamente.
4. Devuelve `{ resumen, linkedin }` con un esquema fijo (tool/ `json_schema`):
 - `resumen` : 1–2 frases fieles al one pager (qué hace, para quién, diferenciador). Prohibido inventar cifras o enfoques no presentes en la fuente.
 - `linkedin` : "[autores] presentaron [Proyecto], [descripción fiel]. [cierre que se desprende del one pager]. " + HT .
5. El resultado se guarda en la tabla `proyecto` con `descripcion_aprobada = false` y `onepager_sha256` . El coordinador lo revisa y aprueba. **Solo se regenera si cambia el hash del one pager** (caché por fuente) — así no se reprocesa ni se machaca lo aprobado.

```
// supabase/functions/generar-descripciones/index.ts (Deno) — esqueleto
// Reusa el patrón de 'extraer-calendario' (§17.4): misma key secreta, mismo modelo Opus.
// content = [{type:'document'|'image', source:{...}}, {type:'text', text: prompt}]
// tools: [{ name:'descripcion', input_schema:{ resumen:string, linkedin:string } }]
// tool_choice: { type:'tool', name:'descripcion' }
// model: 'claude-opus-4-8' // visión + calidad; sin límite de tiempo en Edge Functions
```

Regla dura: si no hay one pager legible para un proyecto, la función devuelve error y el build/publicación **no continúa** para ese proyecto. Nunca se cae a un texto genérico. (§5.4, Paso 5,

17.5 RLS (Row Level Security) — el control de acceso

Activar RLS en cada tabla y definir políticas. Ejemplos:

```
alter table proyecto enable row level security;

-- Cualquiera puede leer proyectos publicados (para el index interno / portal participant
create policy "lectura proyectos publicados"
  on proyecto for select using (estado = 'publicado');

-- Solo coordinador/super_admin de la cohorte pueden escribir
create policy "coordinador escribe"
  on proyecto for all
  using (auth.jwt() -> 'app_metadata' ->> 'app_role' in ('coordinador','super_admin'));
```

Los tokens de panelista/participante (§16.8, §16.16) se validan en una Edge Function o con políticas que comparen contra la tabla `token_*`.

17.6 Storage (logos, one pagers, PDFs)

Buckets `logos` (público) y `documentos` (privado, con URL firmada para one pagers/business plans). Subida desde el portal de entrega del estudiante (§16.3); las páginas leen por URL pública o firmada.

```
const { data } = sb.storage.from('logos').getPublicUrl(`${cohorte}/${proyecto}.jpg`);
```

17.7 Orden de migración sugerido

1. **Crear el esquema** (§16.4) + RLS + buckets de Storage. Cargar la cohorte de ejemplo (FS 24-26) como seed.
2. **Edge Function `extraer-calendario`** (§17.4) — desbloquea lo de §10. Probar con el PDF real.
3. **Edge Function `generar-descripciones`** (§17.4b) — resúmenes y posts por IA desde el one pager. Sustituye el `SUMMARIES_DATA` hardcodeado del script y cierra el incidente del 16 jun 2026 (§5.4, Paso 5). Sembrar el caché con las descripciones ya aprobadas en producción para no reprocesarlas.
4. **Migrar lecturas:** que `index.html` y el portal de panelistas lean de Supabase en vez de JSONbin (cambiar el `fetch` por `sb.from(...).select()`).
5. **Migrar escrituras + Auth:** admin y programador usan login de Supabase y escriben en tablas; retirar la contraseña `Mba2026` y los bins de JSONbin.
6. **Construir** el calculador de 13 hitos y el generador de sitio de cohorte sobre este backend (§10.5).
7. Cuando todo esté en la plataforma, dar de baja el sitio Netlify o redirigirlo (§16.6.5).

Documento generado el 9 de junio de 2026 — última actualización: 16 de junio de 2026 (§5.4 incidente de descripciones inventadas y requisito obligatorio: resúmenes/posts siempre por IA leyendo la fuente real, sea cual sea su formato, nunca hardcodeados; §17.4b Edge Function `generar-descripciones` ;

§17.7 orden de migración actualizado). Actualizaciones previas (15 jun): §9 programador; §10 Generador de Cohorte y su pausa por límite de Netlify; §17 guía de migración a Supabase; §16.6.6 API real de naves-inalde.com. Sistema desarrollado para NAVES — Executive MBA de INALDE Business School (modalidades Intensivo y Fin de Semana). Documentado con la cohorte de ejemplo NAVES 2026 · Fin de Semana 24-26.